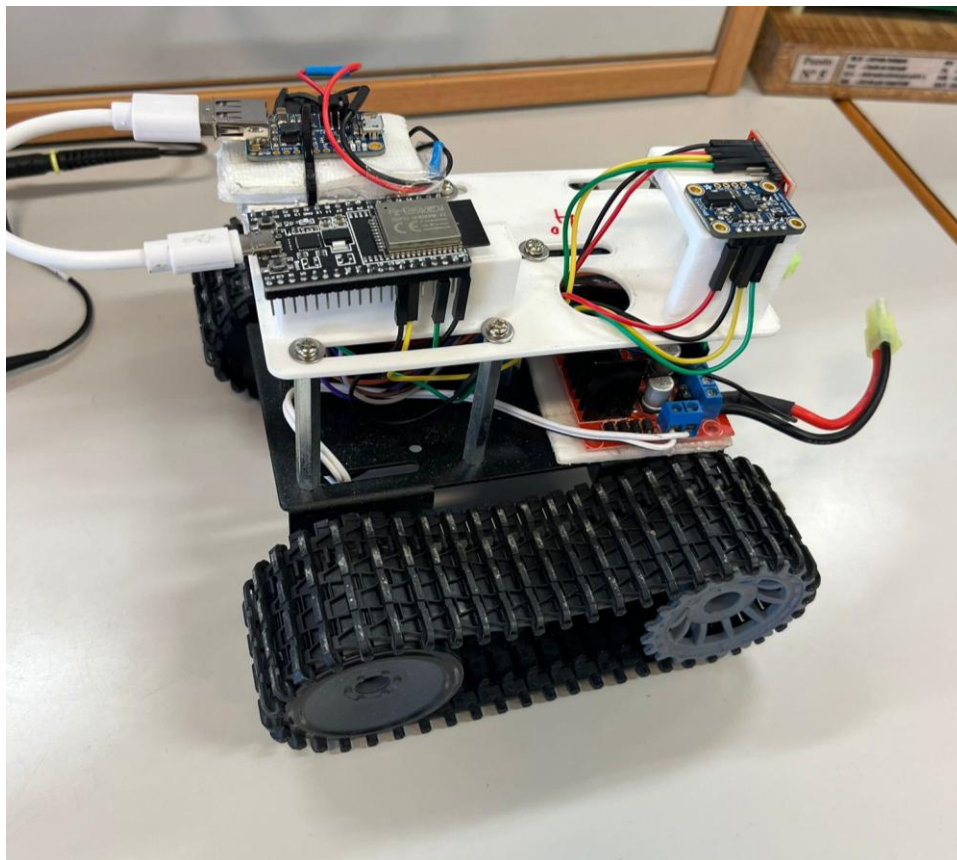


Tecnología Electrónica

Memoria y Desarrollo Proyecto Rover Autónomo



ROMÁN GALIANA ROMERO
GABRIEL LOZANO LÓPEZ

GRADO EN INGENIERÍA AEROESPACIAL-ETSIADI-UPV

Índice

1.	Introducción	3
2.	Descripción del Hardware	3
2.1	Plataforma Móvil	3
2.2	Microcontrolador	4
2.3	Sensores	5
2.4	Actuadores y Control de Potencia.....	6
2.5	Sistema de Alimentación	7
3.	Desarrollo y descripción del Software.....	8
3.1	Primera competición	8
3.2	Segunda prueba: control PID para el guiado autónomo del rover	18
3.3	Tercera prueba	30
4.	Mejoras Proyecto Rover	53
5.	Fuentes utilizadas para el desarrollo del proyecto	54
6.	Conclusiones	55

1. Introducción

Este proyecto ha sido realizado por Román Galiana Romero (rgalrom@etsiadi.upv.es) y Gabriel Lozano López (glozlop@etsiadi.upv.es).

El objetivo de este proyecto ha consistido en implementar todos los conocimientos aprendidos en la asignatura de tecnología electrónica para cumplir con los objetivos que se exigían en cada una de las competiciones que se han realizado. Además, a medida que avanzaban las competiciones la dificultad se aumentaba y se requería de mayor precisión en los ensayos y en los códigos.

La primera competición era una prueba en la que había que implementar un código que permitiese el control manual de un Rover por medio de la aplicación Kodular, en la cual se debía dirigir al Rover para recorrer un circuito en el menor tiempo posible.

En el resto de las pruebas ya no se controlaba manualmente el rover, sino que se debía programar un código que permitiese un autoguiado del Rover mediante un control PID entre otras cosas; para que recorra una distancia en línea recta, en el caso de la segunda competición, o para que esquivе obstáculos y llegué a la meta sin chocarse en el caso de la tercera y última competición.

2. Descripción del Hardware

A continuación, se detalla la relación de componentes hardware utilizados en el desarrollo del rover, incluyendo sus especificaciones técnicas principales, precio aproximado de mercado y proveedores habituales.

2.1 Plataforma Móvil

Chasis Rover Black Gladiator (DFRobot)

- Descripción: Plataforma robótica tipo "tanque" con orugas de alta tracción, diseñada para ofrecer estabilidad y maniobrabilidad en terrenos irregulares. Su estructura de aleación de aluminio proporciona robustez manteniendo un peso contenido.
- Características Principales:
 - Dimensiones: 193mm x 163mm x 60mm.
 - Sistema de tracción: Orugas con motores DC de alto par.
 - Compatibilidad: Diseñado para integrar fácilmente placas como Arduino o Raspberry Pi.

- Precio Aproximado: 70€ - 80€.
- Dónde comprar: [DFRobot](#), RobotShop.

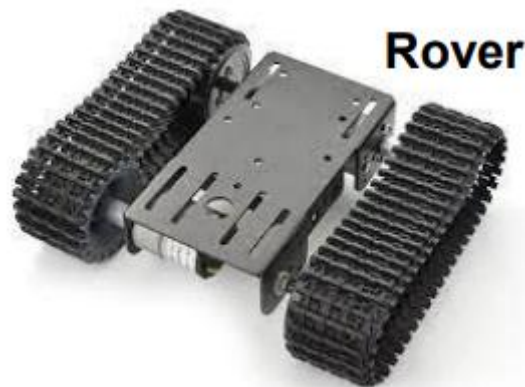


Figura 2.1: Chasis Rover

2.2 Microcontrolador

Microcontrolador: ESP32 DevKit V1

Como núcleo del sistema de control del rover, se ha seleccionado el microcontrolador de alto rendimiento ESP32, específicamente el módulo ESP32-WROOM-32 integrado en una placa de desarrollo modelo DevKit V1.

Su elección se fundamenta en su arquitectura de 32 bits, que ofrece un poder de procesamiento y una capacidad de memoria muy superiores a las plataformas de 8 bits convencionales, características indispensables para gestionar la cinemática del rover y el procesamiento de datos en tiempo real.

- **Procesador de Alto Rendimiento:** Cuenta con un microprocesador (μ C) Dual-Core de 32 bits (Xtensa® LX6), lo que permite ejecutar tareas complejas de navegación y comunicación simultáneamente sin latencia.
- **Voltaje de Operación:** Funciona con una lógica de 3.3V, estándar en sensores modernos, aunque la placa incluye un regulador para alimentarse externamente con rangos superiores (5V-12V) a través del pin VIN.
- **Memoria Ampliada:** Dispone de una capacidad de memoria significativamente mayor que otros microcontroladores estándar, con 520 KB de SRAM y 4 MB de memoria Flash, permitiendo almacenar códigos extensos y buffers de datos grandes para la transmisión de mapas.
- **Conectividad y Periféricos:** Destaca por su versatilidad en comunicaciones, ofreciendo múltiples canales UART e I2C (utilizados para el sensor láser y la IMU), así como conectividad inalámbrica Wi-Fi y Bluetooth integrada.

- Precio aproximado: 8,00 € - 12,00 €.
- Disponibilidad: Tiendas de electrónica online (Amazon, componentes electrónicos), habitualmente vendido bajo marcas como AZDelivery o fabricantes genéricos.

2.3 Sensores

Sensor de Distancia Matricial ToF (SparkFun VL53L5CX)

- Descripción: Sensor de tiempo de vuelo (Time-of-Flight) avanzado que permite medir distancias en múltiples zonas simultáneamente (matriz de 8x8 zonas). A diferencia de los sensores ultrasónicos o IR simples, ofrece un mapa de profundidad 3D básico del entorno.
- Características Principales:
 - Rango de medición: Hasta 4 metros.
 - Resolución: Matriz de 8x8 (64 zonas) o 4x4 (16 zonas).
 - Interfaz: I2C (Qwiic).
- Precio Aproximado: 25€ - 30€.
- Dónde comprar: SparkFun, DigiKey, Mouser.



VL53L5CX

Figura 2.2: Sensor distancia matricial

Unidad de Medida Inercial (IMU) BNO055 (Adafruit)

- Descripción: Sensor de 9 grados de libertad (9-DOF) que integra acelerómetro, giroscopio y magnetómetro. Su característica más destacada es que incluye un procesador ARM Cortex-M0 interno que realiza la fusión de sensores,

entregando datos de orientación (grados de Euler o cuaterniones) listos para usar, liberando al microcontrolador principal de cálculos complejos.

- Características Principales:
 - Fusión de sensores inteligente (Absolute Orientation).
 - Interfaces: I2C y UART.
 - Salida de datos: Orientación, aceleración lineal, gravedad, etc.
- Precio Aproximado: 35€ - 40€.
- Dónde comprar: Adafruit, Mouser, DigiKey.



Figura 2.3: IMU

2.4 Actuadores y Control de Potencia

Driver de Motores L298N

- Descripción: Módulo controlador de doble puente en H basado en el chip L298N. Permite controlar la dirección y velocidad de dos motores de corriente continua de forma independiente mediante señales PWM.
- Características Principales:
 - Voltaje de lógica: 5V.
 - Voltaje de operación (motores): 5V - 35V.
 - Corriente máxima: 2A por canal (pico).
 - Protección: Incluye diodos de protección contra corrientes inversas y disipador de calor.
- Precio Aproximado: 5€ - 8€.

- Dónde comprar: Amazon, tiendas de electrónica general (componente genérico).

2.5 Sistema de Alimentación

Batería LiPo 11.1V 2200mAh (Generic / LIPO SAFE)

- Descripción: Batería de Polímero de Litio de 3 celdas (3S) utilizada para alimentar la etapa de potencia (motores). Proporciona la alta corriente de descarga necesaria para el movimiento del rover.
- Especificaciones: 11.1V, 2200mAh, Alta tasa de descarga (C-rate).
- Precio Aproximado: 20€ - 30€ (incluyendo cargador USB básico).

Adafruit PowerBoost 500 Charger

- Descripción: Convertidor DC/DC elevador (boost converter) que eleva los 3.7V de una batería LiPo a 5.2V estables para alimentar la lógica de control (microcontrolador y sensores). Incluye circuito de carga para la batería.
- Especificaciones: Salida 5V @ 500mA (hasta 1000mA pico).
- Precio Aproximado: 15€ - 20€.
- Batería asociada: LiPo 3.7V 820mAh (~8€ - 12€).
- Dónde comprar: Adafruit, distribuidores oficiales.

3. Desarrollo y descripción del Software

A continuación, se muestra el progreso y el desarrollo de las competencias y como se han implementado los conocimientos adquiridos en clase para su implementación en el software del rover para cada una de las competencias.

3.1 Primera competición

En la primera competición de la asignatura se implementó un sistema de control manual del rover, cuyo objetivo principal era permitir el manejo manual directo del vehículo por durante el recorrido definido en la prueba. Para ello, se desarrolló una aplicación móvil mediante Kodular, desde la cual se le podía enviar órdenes de velocidad y giro al rover de forma inalámbrica. Estas órdenes eran recibidas y procesadas por el ESP32, encargado de gobernar los motores de corriente continua del sistema de tracción.

La arquitectura del sistema se basa en una comunicación Bluetooth entre el smartphone y el ESP32. La aplicación móvil actúa como interfaz hombre-máquina, generando los comandos de control en función de la posición de dos deslizadores virtuales. El ESP32 recibe dichos comandos y los traduce en señales eléctricas adecuadas para el control de los motores a través de un driver de potencia tipo puente H. Este driver permite tanto la regulación de la velocidad de giro como la inversión del sentido de rotación de cada motor, lo que resulta imprescindible para el correcto desplazamiento del rover.

El control de los motores se realiza combinando dos principios fundamentales estudiados en la asignatura: la modulación por ancho de pulsos (PWM) para el control de velocidad y la conmutación de polaridad para el control del sentido de giro. La velocidad de cada motor se ajusta variando el ciclo de trabajo de la señal PWM aplicada a la entrada de habilitación del driver, lo que modifica la tensión media suministrada al motor y, por tanto, su velocidad de giro. Por otro lado, el sentido de rotación se controla mediante señales digitales que determinan la configuración del puente H, permitiendo el giro hacia adelante, hacia atrás o la detención completa del motor.

La estrategia de control manual implementada se basa en el envío de dos comandos independientes desde la aplicación móvil: uno asociado al movimiento longitudinal del rover (avance o retroceso) y otro asociado al giro (izquierda o derecha). A partir de estos dos valores, el ESP32 calcula la consigna de velocidad para cada uno de los motores, siguiendo un esquema de tracción diferencial. De este modo, el rover puede desplazarse en línea recta cuando ambos motores reciben la misma consigna, describir curvas cuando existe una diferencia de velocidad entre las orugas, o incluso girar sobre su propio eje cuando un motor avanza y el otro retrocede simultáneamente.

Este enfoque proporciona una gran maniobrabilidad y facilita el control en espacios reducidos.

El sistema fue diseñado teniendo en cuenta los requisitos específicos del recorrido de la primera competición. En particular, la necesidad de realizar maniobras tipo slalom, la entrada y salida de la zona de maniobra con cambio de sentido, y la obligación de cruzar la meta con el morro del rover orientado hacia delante. La posibilidad de controlar de forma continua tanto la velocidad como el sentido de cada motor permitió adaptar el movimiento del rover a cada tramo del recorrido, realizando correcciones suaves y precisas. Además, este control manual buscaba minimizar impactos con los obstáculos, ya que cada colisión suponía una penalización en el tiempo total de la prueba.

En conjunto, el sistema de control manual desarrollado constituye una base sólida para el resto de las competiciones del proyecto, ya que integra los conceptos fundamentales de control de motores DC, comunicación inalámbrica y control diferencial del movimiento. Este primer enfoque permitió validar el correcto funcionamiento del hardware, del driver de potencia y de la comunicación Bluetooth, sirviendo como punto de partida para la posterior implementación de estrategias de control más avanzadas, como el control automático y la navegación autónoma.

En primer lugar, se optó por el desarrollo de un sistema de control manual del rover mediante una aplicación creada en Kodular, comunicada con el microcontrolador ESP32 a través de Bluetooth. Con ello habilitamos el control directo e intuitivo de la velocidad y el sentido de giro de cada una de las orugas del rover, de forma independiente, utilizando dos barras deslizantes en la interfaz gráfica.

En esta primera aproximación, la aplicación Kodular enviaba dos valores independientes, uno para cada motor del rover (izquierdo y derecho). Cada barra deslizante tomaba valores en el rango 0–200, donde el valor central (100) correspondía al motor parado, valores superiores a 100 indicaban giro hacia adelante y valores inferiores a 100 indicaban giro hacia atrás, ya que al recibirse en Arduino, se le restaba 100, teniendo una señal de -100 a 100. De este modo, el valor absoluto de cada barra representaba el ciclo de trabajo de la señal PWM aplicada al motor, mientras que el signo del valor determinaba el sentido de giro. Este planteamiento permitía, por ejemplo, girar a la derecha incrementando únicamente la velocidad de la oruga izquierda, o realizar giros más cerrados reduciendo o invirtiendo la velocidad de la oruga contraria.

Para implementar este funcionamiento se desarrolló un primer código en Arduino para el ESP32, en el cual el microcontrolador recibía directamente los valores enviados desde Kodular y los aplicaba de forma inmediata a cada motor. El programa se encargaba de interpretar el sentido de giro a partir del signo del valor recibido y de

generar la señal PWM correspondiente, escalando el valor al rango permitido por el sistema de control PWM del ESP32 (desde 0 a 32767). Esta solución era sencilla y directa, y permitió verificar rápidamente el correcto funcionamiento del hardware, del sistema de comunicación Bluetooth y del control básico de los motores.

```
#include <BluetoothSerial.h>

#define ENA 12
#define IN1 14
#define IN2 27

#define ENB 33
#define IN3 26
#define IN4 25

#define CHANNEL1 0
#define CHANNEL2 1
#define FREQ 1000
#define RES 15

BluetoothSerial SerialBT;

void setup() {
  SerialBT.begin("ESP32BT_Rover4");

  ledcAttachChannel(ENA, FREQ, RES, CHANNEL1);
  ledcAttachChannel(ENB, FREQ, RES, CHANNEL2);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);

  ledcWrite(ENA, 0);
  ledcWrite(ENB, 0);

  delay(1000);
}
```

```

void loop() {
  int DutyLeft, DutyRight;
  unsigned int cTonLeft, cTonRight;

  if (SerialBT.available() >= 2) {

    DutyLeft = SerialBT.read() - 100;

    DutyRight = SerialBT.read() - 100;

    if (DutyLeft > 0) {
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
    } else if (DutyLeft < 0) {
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
    } else {
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
    }

    if (DutyRight > 0) {
      digitalWrite(IN3, HIGH);
      digitalWrite(IN4, LOW);
    } else if (DutyRight < 0) {
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, HIGH);
    } else {
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
    }

    cTonLeft = abs(DutyLeft) * 32767 / 100;
    cTonRight = abs(DutyRight) * 32767 / 100;

    ledcWrite(ENA, cTonLeft);
    ledcWrite(ENB, cTonRight);
  }
}

```

Figure 1. Código Arduino del primer intento de la primera competición.

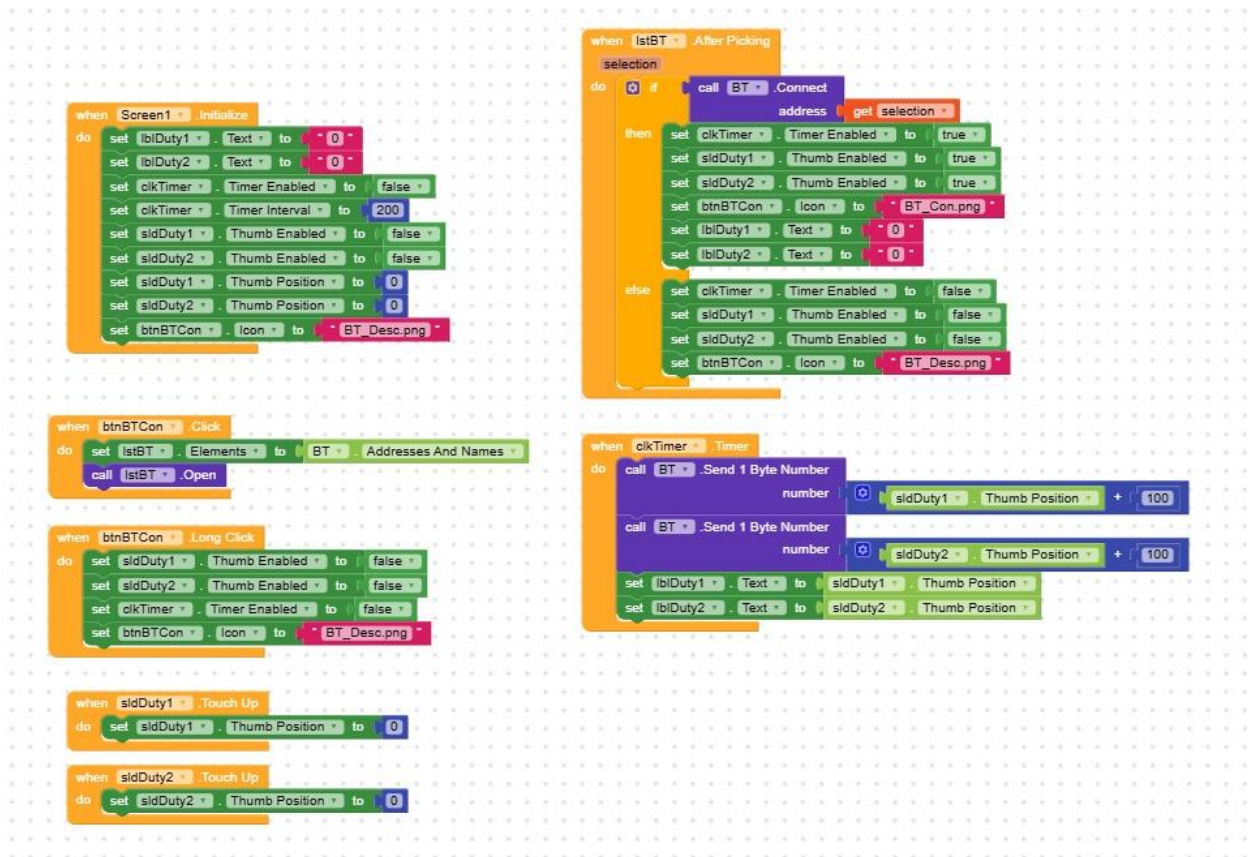


Figure 2. Código de la aplicación móvil desarrollada en Kodular, correspondiente al primer intento de la primera competición.

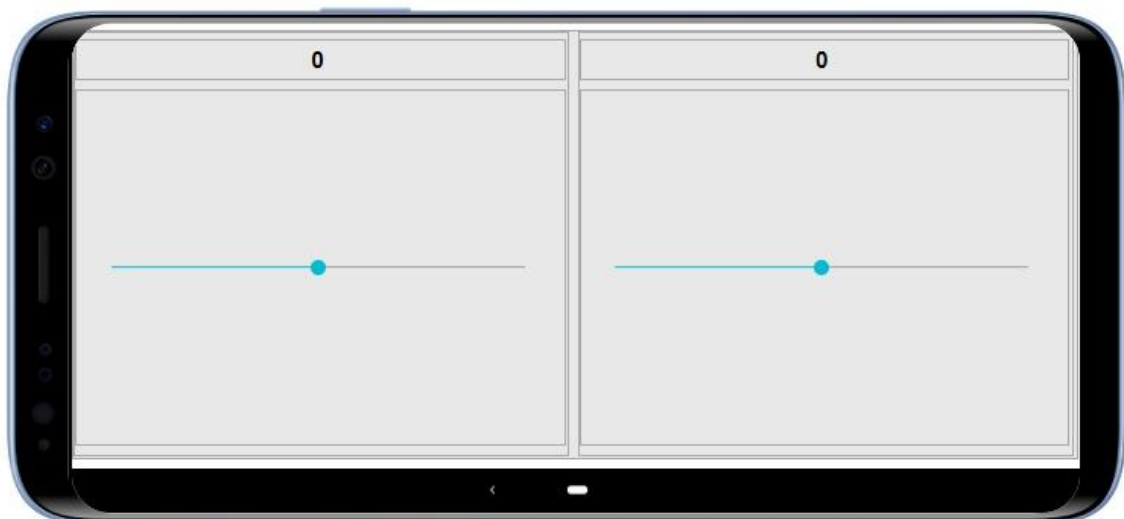


Figure 3. Interfaz de la aplicación móvil desarrollada en Kodular, correspondiente al primer intento de la primera competición.

Tras varias pruebas prácticas con este primer enfoque, se observó que, aunque el control era funcional, requería una coordinación constante a la hora de manejarlo entre ambas barras deslizantes para mantener trayectorias rectas o realizar giros suaves. Esto hacía el manejo menos intuitivo, especialmente en situaciones que requieran ajustes rápidos o precisos durante el recorrido del rover.

Por este motivo, se decidió evolucionar el sistema hacia un segundo enfoque de control manual, también implementado mediante Kodular y Bluetooth, pero basado en el envío de únicamente dos comandos globales: uno asociado a la potencia longitudinal (avance/retroceso) y otro asociado al giro (izquierda/derecha). A partir de estos dos valores, el propio microcontrolador se encargaría de calcular internamente la velocidad de cada oruga mediante un algoritmo de mezcla. Este nuevo planteamiento permitió un control más natural del rover, facilitando el guiado y reduciendo la dificultad a la hora de manejarlo.

En una segunda fase del desarrollo del control manual se implementó un enfoque más avanzado y robusto, manteniendo la comunicación Bluetooth entre la aplicación Kodular y el ESP32, pero modificando completamente la filosofía de control. En lugar de enviar directamente el ciclo de trabajo de cada motor de forma independiente, la aplicación pasó a transmitir únicamente dos comandos globales: uno asociado al movimiento longitudinal del rover (avance/retroceso) y otro asociado al giro (izquierda/derecha). Esta estrategia permitió delegar en el microcontrolador el cálculo de la velocidad individual de cada oruga, simplificando el control desde la aplicación mejorando notablemente la manejabilidad del sistema.

El programa implementado en el ESP32 comienza inicializando la comunicación Bluetooth mediante la librería `BluetoothSerial`, asignando un nombre identificativo al rover para facilitar su emparejamiento con la aplicación Kodular. A continuación, se configuran los pines asociados a los motores izquierdo y derecho, así como los canales PWM del periférico LEDC del ESP32. Cada motor dispone de un canal PWM independiente, lo que permite un control preciso y desacoplado de la velocidad de ambas orugas. Los pines de control de sentido se configuran como salidas digitales y, durante la fase de inicialización, se asegura que los motores permanezcan detenidos para evitar movimientos involuntarios al arrancar el sistema.

Durante la ejecución del programa, el bucle principal espera a recibir dos bytes a través de la conexión Bluetooth. Estos valores representan los comandos enviados desde Kodular: uno correspondiente a la potencia longitudinal (*POWER*) y otro correspondiente al giro (*TURN*), ambos en el rango 0–200. Estos valores se transforman internamente a un rango simétrico de –100 a +100, donde el cero representa la condición de reposo. Esta normalización permite trabajar con magnitudes físicas coherentes, donde el signo indica el sentido del movimiento y el valor absoluto representa la intensidad del mismo.

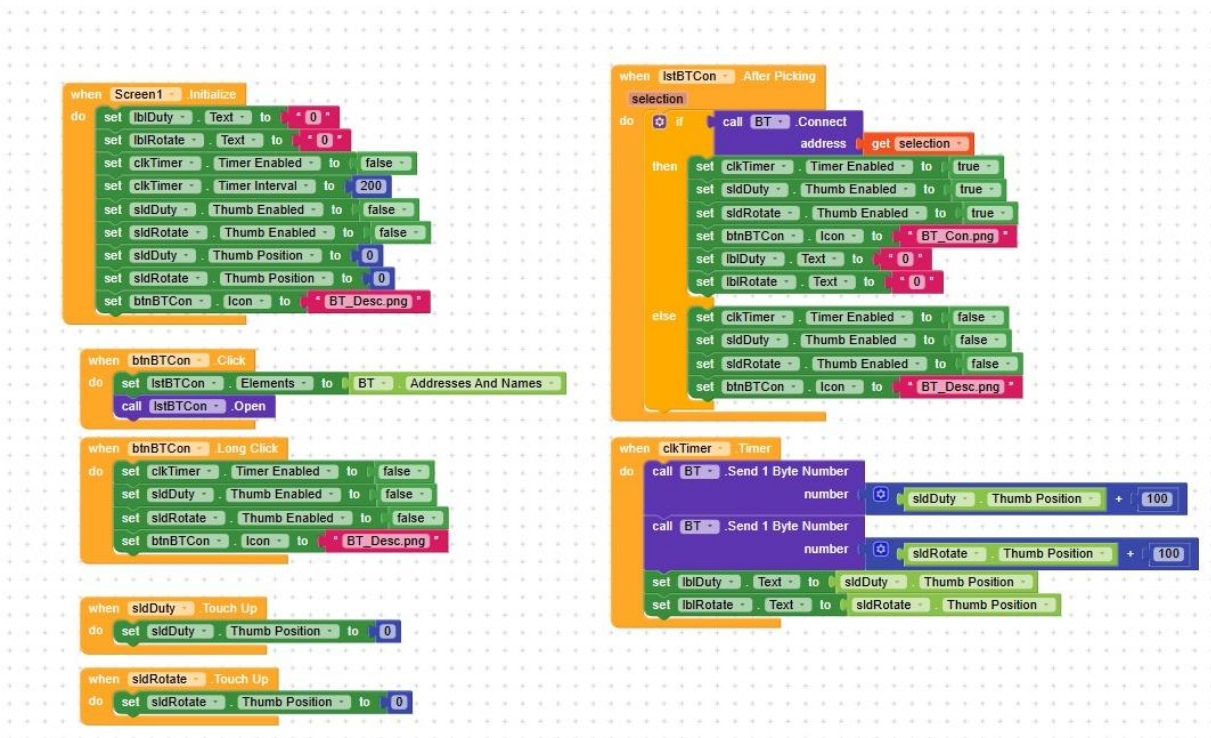


Figure 4. Código final de la aplicación móvil desarrollada en Kodular, correspondiente a la primera competición.

Una vez obtenidos los valores normalizados de potencia y giro, se aplica un algoritmo de mezcla típico de sistemas de tracción diferencial (*skid-steer*). En este esquema, la velocidad de la oruga izquierda se calcula como la suma de la potencia longitudinal y el giro, mientras que la velocidad de la oruga derecha se obtiene como la diferencia entre ambos términos. Este método permite generar de forma natural trayectorias rectas, giros suaves o giros sobre el propio eje, dependiendo de la combinación de los comandos recibidos. Para garantizar que los motores no reciban consignas fuera de su rango operativo, las velocidades calculadas se limitan mediante una función de saturación al intervalo permitido de -100 a $+100$.

A partir del signo de la velocidad de cada oruga se determina el sentido de giro del motor correspondiente, activando adecuadamente los pines digitales de control del puente H. Un valor positivo implica giro hacia adelante, un valor negativo giro hacia atrás y un valor nulo detiene el motor. De forma independiente, el valor absoluto de la velocidad se convierte en un ciclo de trabajo PWM proporcional, escalado al rango de resolución del sistema PWM del ESP32 ($0-32767$). Este valor se envía finalmente al canal PWM asociado a cada motor, controlando así su velocidad de forma continua.

Este segundo enfoque de control manual demostró ser claramente superior al primero durante las pruebas experimentales. Al separar conceptualmente el movimiento longitudinal del giro, el manejo del rover resultó mucho más intuitivo, permitiendo

trayectorias más suaves, mayor precisión en los giros y una reducción significativa de la carga de control desde la aplicación. Además, esta estructura de control sentó las bases para la posterior implementación del control automático del rover, facilitando la integración de algoritmos más avanzados sin necesidad de modificar la arquitectura básica de accionamiento de los motores.

```

#include <BluetoothSerial.h> // Libreria Bluetooth Serial del ESP32

// Definiciones de pines para el Motor A (Izquierdo)
#define ENA 12 // entrada de velocidad (PWM izquierdo)
#define IN1 14 // (control de sentido atrás)
#define IN2 27 // (control de sentido adelante)

// Definiciones de pines para el Motor B (Derecho)
#define ENB 33 // entrada de velocidad (PWM derecho)
#define IN3 26 // (control de sentido adelante)
#define IN4 25 // (control de sentido atrás)

// Configuración del PWM
#define CHANNEL1 0 // izquierdo
#define CHANNEL2 1 // derecho
#define FREQ 1000 // PWM de 1kHz
#define RES 15 // 15 bits (de 0 a 32767)

BluetoothSerial SerialBT;

void setup() {

    SerialBT.begin("ESP32BT_Rover4");

    // control de velocidad independiente para cada motor
    ledcAttachChannel(ENA, FREQ, RES, CHANNEL1); // pin ENA (PWM izquierdo)
    ledcAttachChannel(ENB, FREQ, RES, CHANNEL2); // pin ENB (PWM derecho)

    // pines que controlan el sentido de velocidad configurados como salida
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    // motores detenidos al inicio
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);

    // motores empiezan apagados
    ledcWrite(ENA, 0);
    ledcWrite(ENB, 0);

    delay(1000); // esperar 1s
}

void loop() {

    int kodularPower, kodularTurn; // Kodular envia 2 valores: POWER (0-200) - adelante/atrás ; TURN (0-200)- izquierda/derecha

    // Definimos variables (tipo float para precisión)
    float powerY, turnX;
    float leftSpeed, rightSpeed;

    unsigned int cTonLeft, cTonRight; // PWM final de cada motor (0-32767)

    if (SerialBT.available() >= 2) { // Espera a recibir los 2 bytes (POWER y TURN) del Kodular

        kodularPower = SerialBT.read(); // Valor 0-200 de Kodular
        kodularTurn = SerialBT.read(); // Valor 0-200 de Kodular

        // Ponemos la Potencia de (-100 a +100)
        // 0 -> -100 (Atrás)
        // 100 -> 0 (Parado)
        // 200 -> +100 (Adelante)
        powerY = (float)kodularPower - 100.0;

        // Ponemos el Turn de (-100 a +100)
        // 0 -> -100 (Izquierda)
        // 100 -> 0 (Recto)
        // 200 -> +100 (Derecha)
        turnX = (float)kodularTurn - 100.0;

        // Si TURN = 0 - ambos motores reciben misma potencia (sentido hacia adelante)
        // Si TURN > 0 - motor izquierdo mas rapido (giro a la derecha)
        // Si TURN < 0 - motor derecho mas rapido (giro a la izquierda)
        // Si POWER = 0 && TURN != 0 - una oruga va hacia adelante y la otra hacia atrás
        // → el rover gira sobre su propio eje (giro en el sitio)
        leftSpeed = powerY + turnX;
        rightSpeed = powerY - turnX;

        // limitamos de (-100 a +100) - positivos hacia adelante y negativos hacia atras
        leftSpeed = constrain(leftSpeed, -100.0, 100.0);
        rightSpeed = constrain(rightSpeed, -100.0, 100.0);
    }
}

```

```

// Motor Izquierdo (ENA)
if (leftSpeed > 0) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH); // hacia adelante
} else if (leftSpeed < 0) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW); // hacia atrás
} else {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW); // motor apagado
}

// Motor Derecho (ENB)
if (rightSpeed > 0) {
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW); // hacia adelante
} else if (rightSpeed < 0) {
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH); // hacia atrás
} else {
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW); // motor apagado
}

// conversion a PWM (0-32767) (positivo siempre)
cTonLeft = (unsigned int)(abs(leftSpeed) * 32767.0 / 100.0);
cTonRight = (unsigned int)(abs(rightSpeed) * 32767.0 / 100.0);

// Enviamos PWM a cada motor
ledcWrite(ENA, cTonLeft);
ledcWrite(ENB, cTonRight);
}
}

```

Figure 5. Código Arduino final de la primera prueba.

Este segundo método de control fue finalmente el seleccionado para la primera competición, al ofrecer un comportamiento más estable, predecible e intuitivo, manteniendo además una estructura de software más clara y escalable para futuras ampliaciones del proyecto.

3.2 Segunda prueba: control PID para el guiado autónomo del rover

En la segunda prueba del proyecto se abordó el problema del guiado autónomo del rover a lo largo de un pasillo, con el objetivo de que este fuera capaz de recorrerlo completo sin colisionar con las paredes. Durante las pruebas iniciales se observó que, aun aplicando el mismo valor de PWM a ambos motores, el rover no avanzaba en línea recta, sino que tendía a desviarse progresivamente hacia uno de los lados. Este comportamiento se debe al carácter no ideal del sistema físico, ya que los motores no presentan exactamente las mismas características, la transmisión mecánica no es idéntica en ambos lados y el rozamiento con el suelo no es perfectamente simétrico.

Para corregir estas desviaciones se decidió implementar un sistema de control automático basado en un regulador PID, capaz de compensar en tiempo real cualquier error angular respecto a una referencia de orientación fijada previamente. El objetivo del control es mantener constante el ángulo de orientación del rover mientras avanza, de modo que cualquier giro no deseado sea detectado y corregido automáticamente mediante una acción diferencial sobre las velocidades de las ruedas.

Para conocer la orientación del rover se utilizó una unidad de medida inercial (IMU), concretamente el sensor BNO055. Este dispositivo integra acelerómetro, giróscopo y magnetómetro, y realiza internamente una fusión sensorial que permite obtener directamente la orientación del sistema en forma de ángulos de Euler. De entre estos ángulos se seleccionó el ángulo de yaw (correspondiente a la rotación del rover en el plano horizontal, ya que es la magnitud directamente relacionada con las desviaciones laterales en el pasillo).

sensor proporciona el ángulo de yaw en un rango de 0 a 360 grados, mostrado mediante la letra X. Sin embargo, para facilitar el tratamiento del error en el control, este ángulo se transformó a un rango simétrico de -180 a $+180$ grados, evitando así discontinuidades cuando se cruza el valor de 360 grados.

La referencia del sistema de control se definió de forma experimental. Antes de iniciar el movimiento, el rover se colocó manualmente en el pasillo intentando alinear lo mejor posible con su eje longitudinal. El ángulo de yaw medido en ese instante se tomó como referencia inicial del sistema, considerándose ese valor como el ángulo deseado.

Para facilitar la operación durante las pruebas, se implementó un comando de “reset” accesible desde la aplicación móvil de Kodular, que permitía redefinir la referencia angular en cualquier momento sin necesidad de reiniciar el sistema. De este modo, se podía recalibrar la orientación de referencia si se observaba una desviación acumulada o si cambiaban las condiciones del entorno.

El error del sistema se definió como la diferencia entre el ángulo de referencia y el ángulo actual medido por la IMU, expresado siempre dentro del intervalo -180 a $+180$ grados.

El sistema de control implementado sigue la estructura clásica de un bucle de control en lazo cerrado. La referencia angular se compara con la orientación real del rover para obtener un error angular. Este error se introduce en el regulador PID, cuya salida constituye la acción de control. Dicha acción se aplica al sistema físico, en este caso los motores del rover, generando una corrección en la velocidad de las ruedas que modifica la orientación del vehículo. La nueva orientación es medida de nuevo por la IMU, cerrando así el lazo de realimentación.

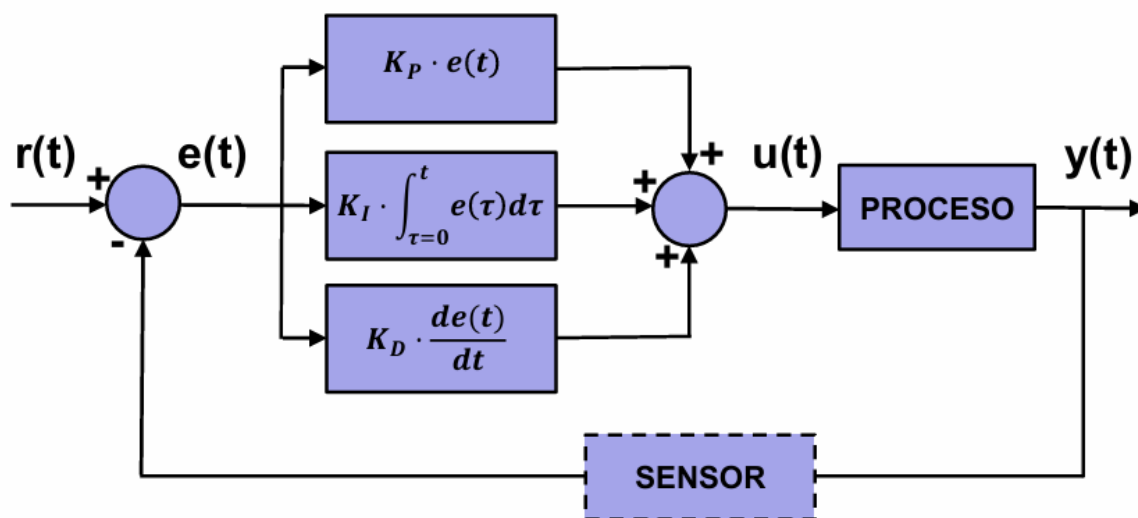


Figure 6. Diagrama de bloques del control PID.

Este planteamiento permite corregir automáticamente cualquier perturbación externa, como pequeñas irregularidades del suelo o diferencias de tracción entre las ruedas.

Se optó por un control PID debido a las limitaciones observadas al emplear estrategias más simples. Un control puramente proporcional permite corregir las desviaciones, pero puede dejar un error estacionario y provocar oscilaciones si la ganancia es elevada. La incorporación de una acción integral permite eliminar este error persistente acumulando el error a lo largo del tiempo, mientras que la acción derivativa contribuye a mejorar la estabilidad del sistema anticipándose a los cambios rápidos del error.

El regulador PID se implementó de forma digital, ya que el control se ejecuta en el microcontrolador. Para ello se discretizaron las acciones integral y derivativa utilizando aproximaciones basadas en sumas y diferencias finitas, empleando el tiempo transcurrido entre iteraciones como periodo de muestreo.

Dado que no se disponía de un modelo matemático preciso del rover, las ganancias del regulador PID se ajustaron de forma experimental. Para ello se desarrolló un primer programa de prueba que permitía modificar en tiempo real las ganancias proporcional, integral y derivativa mediante la aplicación móvil Kodular. Tres deslizadores independientes enviaban los valores de Kp, Ki y Kd al microcontrolador mientras el rover estaba en funcionamiento.

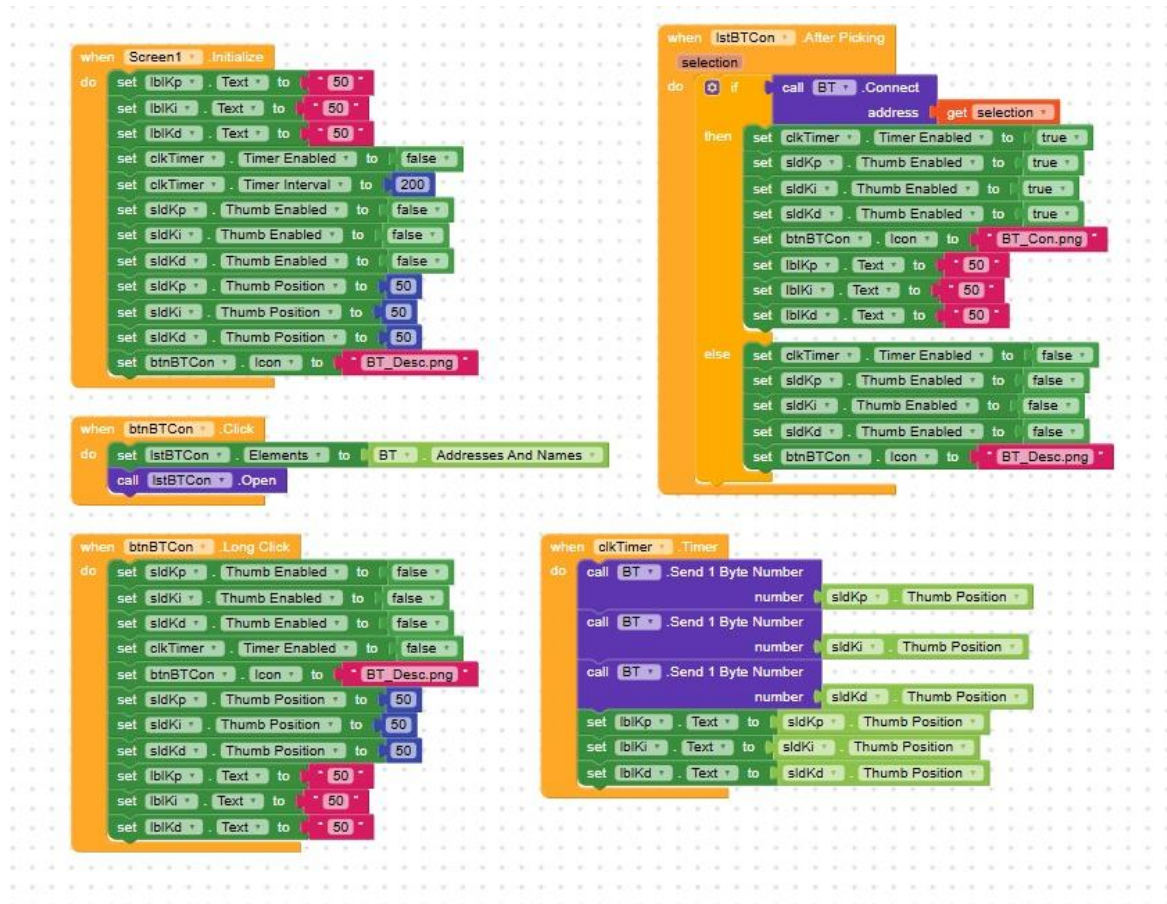


Figure 7. Código de la aplicación móvil desarrollada en Kodular, correspondiente al ajuste experimental de las ganancias del PID.

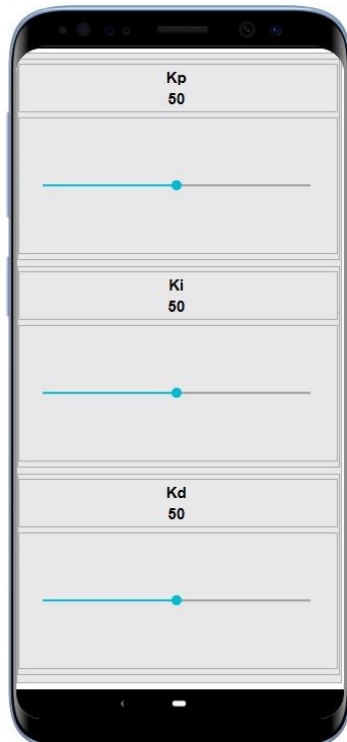


Figure 8. Interfaz de la aplicación móvil desarrollada en Kodular, correspondiente al ajuste experimental de las ganancias del PID.

Este enfoque permitió observar directamente el efecto de cada ganancia sobre el comportamiento del sistema. Durante las pruebas se analizaron aspectos como la rapidez de corrección, la estabilidad del movimiento y la presencia de oscilaciones. De este modo se pudieron seleccionar unos valores de ganancias que ofrecían un compromiso adecuado entre respuesta rápida y estabilidad.

```

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
#include <BluetoothSerial.h>

// ----- IMU (BNO055) -----
Adafruit_BNO055 bno = Adafruit_BNO055(55);
sensors_event_t event; // Para guardar los ángulos de Euler

// ----- ROVER: Pines motores -----
// Definiciones de pines para el Motor A (Izquierdo)
#define ENA 12
#define IN1 14
#define IN2 27

// Definiciones de pines para el Motor B (Derecho)
#define ENB 23
#define IN3 26
#define IN4 25

// PWM
#define CHANNEL1 0 // PWM motor izquierdo
#define CHANNEL2 1 // PWM motor derecho
#define FREQ 1000 // PWM kHz
#define RES 15 // 15 bits → 0..32767

BluetoothSerial SerialBT;

// ----- Variables PID -----
// Ganancias (inicializamos a 0, las cambiaremos desde Kodular)
float Kp = 0.0; // proporcional
float Ki = 0.0; // integral
float Kd = 0.0; // derivativa

// Variables del PID
float ref = 0.0; // ángulo de referencia Z = 0°
float X; // Variable a controlar (ángulo de Euler Z)
float error; // e(k)
float prev_error = 0; // e(k-1)
long T; // tiempo actual (micros)
long prev_T; // tiempo anterior (micros)
float delta_T; // Ts = T - prev_T (en segundos)
float integral = 0; // suma de errores (parte I)
float derivator = 0; // derivada del error (parte D)
float u = 0; // salida del PID (acción de control)

// Límites
const float INTEGRAL_MAX = 100.0; // límite de la integral
const float U_MAX = 100.0; // límite de la salida u (como "porcentaje extra")

const float BASE_SPEED = 100.0; // % de duty base hacia adelante

```

```

// ----- FUNCIONES AUXILIARES -----

// Convierte duty (0-100) a PWM (0-32767)
unsigned int dutyToPWM(float dutyPercent) {
    if (dutyPercent < 0)    dutyPercent = 0;
    if (dutyPercent > 100) dutyPercent = 100;
    return (unsigned int)(dutyPercent * 32767.0 / 100.0);
}

// Lee las ganancias PID que llegan desde Kodular (3 sliders: Kp, Ki, Kd)
void leerGananciasDesdeBT() {
    // esperamos que hayan 3 bytes en el buffer (recibidos de Kodular)
    if (SerialBT.available() >= 3) {
        uint8_t kpByte = SerialBT.read(); // 0-100
        uint8_t kiByte = SerialBT.read(); // 0-100
        uint8_t kdByte = SerialBT.read(); // 0-100

        Kp = kpByte / 10.0;
        Ki = kiByte / 10.0;
        Kd = kdByte / 10.0;

        Serial.print(" Kp = "); Serial.print(Kp);
        Serial.print(" Ki = "); Serial.print(Ki);
        Serial.print(" Kd = "); Serial.println(Kd);
    }
}

// ----- SETUP -----
void setup() {

    Serial.begin(115200);

    // --- IMU ---
    Wire.begin();
    Wire.setClock(400000);

    if (!bno.begin()) {
        Serial.println("BNO055 no detectado. Comprobar conexiones o direcci3n I2C");
        while (1); // bloqueado aqu3 si no hay IMU
    }
    bno.setExtCrystalUse(true);
    delay(1000);

    SerialBT.begin("ESP32BT_Rover4");

    // --- Motores (PWM + sentido) ---
    ledcAttachChannel(ENA, FREQ, RES, CHANNEL1); // motor izquierdo
    ledcAttachChannel(ENB, FREQ, RES, CHANNEL2); // motor derecho

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    // Ambos motores parados al inicio
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    ledcWrite(ENA, 0);
    ledcWrite(ENB, 0);

    // --- PID ---
    prev_T = micros();
    integral = 0;
    prev_error = 0;
}

// ----- LOOP -----
void loop() {

    // 1) Leer (si llegan) nuevas ganancias Kp, Ki, Kd desde Kodular
    leerGananciasDesdeBT();

    // 2) Leer IMU → 3ngulos de Euler
    bno.getEvent(&event);
    X = event.orientation.x; // eje x = yaw (giro derecha/izquierda)

    // Convertir 0-360 a -180..+180
    if (X > 180) {
        X = X - 360;
    }

    // calculamos error
    ref = 0.0;
    error = ref - X; // e = 0 - X

    // 4) Calcular delta_T (en segundos)
    T = micros();
    delta_T = (float)(T - prev_T) / 1.0e6; // micros → segundos

    // Parte integradora del PID
    integral += error * delta_T;

    // Saturaci3n anti-windup de la integral
    if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
    if (integral < -INTEGRAL_MAX) integral = -INTEGRAL_MAX;
}

```

```

// Parte derivativa del PID
derivator = (error - prev_error) / delta_T;

// Actualizar valores para la próxima iteración
prev_T = T;
prev_error = error;

// Salida PID:  $u = K_p \cdot e + K_i \cdot \int e + K_d \cdot de/dt$ 
u = Kp * error + Ki * integral + Kd * derivator;

// Saturar la salida u
if (u > U_MAX) u = U_MAX;
if (u < -U_MAX) u = -U_MAX;

// - Tomamos una velocidad base (BASE_SPEED) hacia adelante en las dos ruedas.
// - Añadimos +u a la izquierda y -u a la derecha segun nuestro control PID:

// EJ: Si el rover gira hacia la derecha (X>0 - error(0-X)<0 - u<0):
// leftSpeed < base, rightSpeed > base → gira hacia la izquierda para corregir.

float leftSpeed = BASE_SPEED + u;
float rightSpeed = BASE_SPEED - u;

// Limitamos a [0,100]
if (leftSpeed < 0) leftSpeed = 0;
if (leftSpeed > 100) leftSpeed = 100;
if (rightSpeed < 0) rightSpeed = 0;
if (rightSpeed > 100) rightSpeed = 100;

// sentido siempre hacia adelante
digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH); // motor izquierdo hacia delante
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW); // motor derecho hacia delante

// Aplicar PWM (0-255) según las velocidades calculadas (aseguramos que son positivas)
unsigned int cTonLeft = dutyToPWM(leftSpeed);
unsigned int cTonRight = dutyToPWM(rightSpeed);

ledcWrite(ENA, cTonLeft);
ledcWrite(ENB, cTonRight);

Serial.print("X = "); Serial.print(X);
Serial.print(" error = "); Serial.print(error);
Serial.print(" u = "); Serial.print(u);
Serial.print(" L = "); Serial.print(leftSpeed);
Serial.print(" R = "); Serial.println(rightSpeed);

// Tiempo de muestreo
delay(50); // 50 ms (20 Hz)
}

```

Figure 9. Código Arduino del ajuste experimental de las ganancias PID.

Tras las pruebas experimentales se determinaron unos valores finales para las ganancias del regulador PID, 7.5 para la ganancia proporcional, 6 para la integral y 0 para la derivativa. Se observó que una ganancia proporcional suficientemente alta permitía corregir eficazmente las desviaciones angulares, mientras que la ganancia integral era necesaria para eliminar pequeños errores persistentes debidos a asimetrías del sistema. La ganancia derivativa no se incluyó en el control final y se fijó a valor cero, ya que su utilización provocaba un incremento notable de las oscilaciones del sistema. Este comportamiento se atribuyó principalmente a la sensibilidad de la acción derivativa al ruido presente en la medida del ángulo proporcionada por la IMU, lo que generaba variaciones bruscas en la señal de control sin aportar ninguna mejora en la estabilidad del movimiento.

Además, se implementaron mecanismos de saturación tanto para la acción integral como para la salida total del controlador, evitando así problemas de acumulación excesiva de error (fenómeno de windup) y asegurando que la acción de control se mantuviera dentro de límites seguros para los motores y no generará oscilaciones en el rover.

La acción de control generada por el PID se aplicó de forma diferencial sobre las ruedas del rover. Se estableció una velocidad base constante hacia adelante para ambos motores, y la salida del regulador se sumó a la velocidad de una rueda y se restó a la de la otra. De esta manera, cuando el rover se desviaba hacia un lado, el control aumentaba la velocidad de la rueda correspondiente a ese lado y reducía la de la opuesta, generando un giro corrector que tendía a restablecer la orientación deseada.

Este enfoque permite realizar correcciones suaves y continuas sin detener el avance longitudinal del rover, lo que resulta especialmente adecuado para recorrer pasillos largos de forma estable, y llegar en el menor tiempo posible.

El sistema incorporó un control básico de estados mediante comandos enviados por Bluetooth desde Kodular. Un comando de inicio/parada permitía activar o desactivar el control PID, garantizando que los motores permanecieran apagados cuando el sistema no estuviera en funcionamiento. Este modo resultó especialmente útil para poder desplazar y orientar manualmente el rover con el control desactivado, permitiendo leer nuevos ángulos de la IMU con el vehículo completamente detenido. Una vez establecida la orientación deseada, el control podía activarse de nuevo mediante el comando de *start*, iniciándose entonces el movimiento de los motores bajo el control del regulador PID. Por su parte, el comando de *reset* se utilizó exclusivamente para redefinir la referencia angular del sistema, sin detener el rover ni reiniciar el programa. Esto permitió modificar la orientación de referencia en cualquier momento, posibilitando recolocar el rover dentro del pasillo y establecer una nueva referencia de *yaw* de forma rápida y cómoda, sin necesidad de apagar y encender el sistema. De este modo, el control podía adaptarse dinámicamente a la posición inicial del rover en cada intento

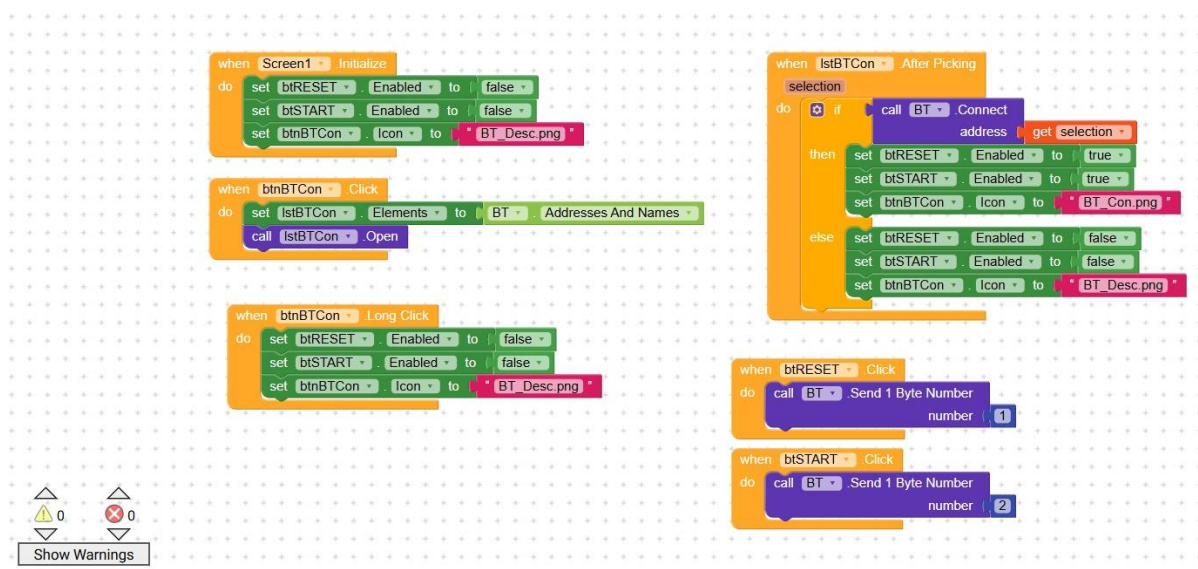


Figure 10. Código final de la aplicación móvil desarrollada en Kodular, correspondiente a la segunda competición.



Figure 11. Interfaz final de la aplicación móvil desarrollada en Kodular, correspondiente a la primera competición.

Una vez determinadas las ganancias óptimas, se desarrolló el programa final de control PID, en el que las ganancias se fijaron como constantes y se eliminaron las funcionalidades de ajuste en tiempo real. Este código implementa el bucle completo de control: lectura de la IMU, cálculo del error angular, ejecución del regulador PID discreto y aplicación de la acción de control sobre los motores.

```

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
#include <BluetoothSerial.h>

// ----- IMU (BNO055) -----
Adafruit_BNO055 bno = Adafruit_BNO055(55);
sensors_event_t event;

// ----- Pines del ROVER -----
#define ENA 12
#define IN1 14
#define IN2 27

#define ENB 33
#define IN3 26
#define IN4 25

// PWM
#define CHANNEL1 0
#define CHANNEL2 1
#define FREQ 1000
#define RES 15

BluetoothSerial SerialBT;

// ----- PID (CONSTANTES FINALES) -----
float Kp = 7.5; // Proporcional
float Ki = 6.0; // Integral
float Kd = 0.0; // Derivativa

// ----- Variables PID -----
float ref = 0.0; // ponemos la referencia inicial 0 (la iremos cambiando con el boton de reset de Modular)
float X;
float error;
float prev_error = 0;

long T;
long prev_T;
float delta_T;

float integral = 0;
float derivator = 0;
float u = 0;

bool started = false; // estado del START/STOP

// Límites
const float INTEGRAL_MAX = 100.0;
const float U_MAX = 100.0;

const float BASE_SPEED = 100.0;

// ----- Función duty-PWM -----
unsigned int dutyToPWM(float dutyPercent) {
  if (dutyPercent < 0) dutyPercent = 0;
  if (dutyPercent > 100) dutyPercent = 100;
  return (unsigned int)(dutyPercent * 32767.0 / 100.0);
}

// ----- PROCESAR COMANDOS BT -----
void leerComandoBT() {
  while (SerialBT.available() > 0) {

    int comando = SerialBT.read(); // <-- recibe 1 o 2

    if (comando == 1) {
      bno.getEvent(&event);
      X = event.orientation.x;
      // RESET
      ref = X;
      integral = 0;
      prev_error = 0;

      Serial.println(">>> RESET (1) recibido");
    }

    else if (comando == 2) {
      // START/STOP
      started = !started;

      if (!started) {
        // Apagar motores
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, LOW);
        ledcWrite(ENA, 0);
        ledcWrite(ENB, 0);

        Serial.println(">>> STOP (2) recibido");
      } else {
        Serial.println(">>> START (2) recibido");
      }
    }
  }
}

// ----- SETUP -----
void setup() {
  Serial.begin(115200);

  // IMU
  Wire.begin();
  Wire.setClock(400000);

```

```

if (!bno.begin()) {
  Serial.println("BNO055 no detectado. Revisar conexiones.");
  while (1);
}
bno.setExtCrystalUse(true);
delay(1000);

// Motores
ledcAttachChannel(ENA, FREQ, RES, CHANNEL1);
ledcAttachChannel(ENB, FREQ, RES, CHANNEL2);

pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

ledcWrite(ENA, 0);
ledcWrite(ENB, 0);

// Bluetooth
SerialBT.begin("ESP32BT_Rover4");

// PID
prev_T = micros();
integral = 0;
prev_error = 0;
}

// ----- LOOP -----
void loop() {

  // 1) Leer comandos: RESET (1) / START-STOP (2)
  leerComandosBT();

  // Si no está iniciado → motores off
  if (!started) {
    delay(20);
    return;
  }

  // 2) Leer IMU
  bno.getEvent(&event);
  X = event.orientation.x;

  float diff = X - ref; // nuestro angulo va a ser este
  // pues cuando le damos al boton de reset el angulo que lee en esos momentos va ser su referencia
  // (nuestro angulo 0)
  if (diff > 180) diff = diff - 360;
  if (diff < -180) diff = diff + 360;

  // 3) PID x
  error = -diff; // ref - x (pero de -180 a 180)

  T = micros();
  delta_T = (float)(T - prev_T) / 1e6;

  integral += error * delta_T;
  if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
  if (integral < -INTEGRAL_MAX) integral = -INTEGRAL_MAX;

  derivator = (error - prev_error) / delta_T;

  prev_T = T;
  prev_error = error;

  u = Kp * error + Ki * integral + Kd * derivator;

  if (u > U_MAX) u = U_MAX;
  if (u < -U_MAX) u = -U_MAX;

  // 4) CONTROL DE MOTORES
  float leftSpeed = BASE_SPEED + u;
  float rightSpeed = BASE_SPEED - u;

  if (leftSpeed < 0) leftSpeed = 0;
  if (leftSpeed > 100) leftSpeed = 100;
  if (rightSpeed < 0) rightSpeed = 0;
  if (rightSpeed > 100) rightSpeed = 100;

  // Siempre adelante
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  ledcWrite(ENA, dutyToPWM(leftSpeed));
  ledcWrite(ENB, dutyToPWM(rightSpeed));

  // Debug
  Serial.print(" X absoluta = "); Serial.print(X);
  Serial.print(" X = "); Serial.print(diff); // la X de verdad en el sistema de coordenadas del Rover con la referencia
  Serial.print(" e = "); Serial.print(error);
  Serial.print(" u = "); Serial.print(u);
  Serial.print(" L = "); Serial.print(leftSpeed);
  Serial.print(" R = "); Serial.println(rightSpeed);
}

```

Figure 12. Código Arduino final de la segunda prueba.

Durante la ejecución de la segunda prueba se comprobó que el rover era capaz de recorrer el pasillo de forma autónoma sin colisionar con las paredes, corrigiendo automáticamente las desviaciones de orientación que aparecían durante el movimiento. No obstante, se observaron algunas limitaciones, como por ejemplo la necesidad de una correcta inicialización de la referencia angular con un buen centrado del rover en el pasillo. A pesar de ello, los resultados obtenidos validan la eficacia del control PID implementado para el guiado autónomo del rover.

3.3 Tercera prueba

La tercera y última prueba de la asignatura consistió en la navegación autónoma de un rover a lo largo de un pasillo con presencia de obstáculos, con el objetivo de alcanzar el extremo opuesto en el menor tiempo posible y evitando colisiones. El sistema penalizaba los impactos con obstáculos, por lo que el diseño debía equilibrar adecuadamente rapidez y seguridad. El rover partía desde una posición inicial conocida y debía tomar decisiones de rumbo y velocidad de manera autónoma, sin intervención manual durante la ejecución.

Para abordar este problema se planteó una estrategia global basada en la combinación de sensado mediante el sensor de distancia matricial ToF, estimación de orientación mediante la IMU, procesamiento de alto nivel en MATLAB y control de bajo nivel ejecutado en el ESP32. MATLAB se encargó de analizar el entorno a partir de la matriz de distancias, decidir el rumbo y la velocidad adecuados y enviar comandos al microcontrolador mediante Bluetooth, mientras que el ESP32 ejecutó el control PID de rumbo y el control directo de los motores.

%% Aquí se recomienda incluir una imagen o diagrama del recorrido de la competición, mostrando el pasillo, la posición de los obstáculos y el sentido de avance del rover. También resulta conveniente añadir una fotografía del rover con los sensores montados, indicando la posición del sensor ToF y de la IMU sobre el chasis. %%

Arquitectura del sistema y reparto de tareas

El sistema se estructuró en dos niveles claramente diferenciados. El nivel de bajo nivel se implementó en el ESP32 y se encargó de la adquisición de datos de los sensores, la ejecución del control PID de rumbo y el control de los motores mediante señales PWM. Además, el ESP32 gestionó la comunicación Bluetooth, recibiendo comandos desde MATLAB y enviando tramas de datos del sensor ToF.

El nivel de alto nivel se implementó en una aplicación desarrollada con MATLAB App Designer. Esta aplicación recibió las tramas de distancia del sensor ToF, reconstruyó la matriz espacial de distancias, aplicó distintos filtrados y criterios de decisión y calculó los comandos de dirección y velocidad que debían enviarse al ESP32. De este modo, se desacopló la lógica de decisión compleja del control en tiempo real de los motores.

La comunicación entre ambos niveles se realizó mediante Bluetooth utilizando mensajes estructurados. Los comandos de control se enviaron en tramas de dos bytes, donde el primer byte indicaba el tipo de comando y el segundo contenía el dato asociado. Por su parte, las medidas del sensor ToF se enviaron en tramas de 160 bytes que incluían tanto las distancias como los estados de cada celda.

%% Aquí se recomienda incluir un diagrama de bloques del sistema completo, mostrando el flujo “sensor ToF → ESP32 → Bluetooth → MATLAB → Bluetooth → ESP32 → motores”, así como una captura de la interfaz de la aplicación MATLAB donde se visualizan las matrices de distancia y estado junto con el botón de inicio/parada. %%

Sensor ToF VL53L5CX: principio de medida y configuración

El sensor VL53L5CX es un sensor de tipo Time of Flight (ToF) que mide distancias mediante la emisión de pulsos de luz láser infrarroja y el cálculo del tiempo que tarda la señal en reflejarse en los objetos y regresar a él. A diferencia de sensores puntuales, este dispositivo proporciona una medida matricial del entorno, dividiendo su campo de visión en una matriz de zonas independientes.

En este proyecto el sensor se utilizó en su resolución nativa de 8×8 zonas, lo que proporciona un total de 64 mediciones de distancia por adquisición. El funcionamiento se realizó en modo continuo, con una frecuencia de adquisición de 10 Hz, suficiente para la velocidad de desplazamiento del rover. El procesamiento interno del sensor incluye un procesamiento espacial interno (sharpener) que mejora la definición entre zonas adyacentes.

Aunque el sensor ofrece un campo de visión amplio, no todas las zonas resultan igualmente útiles para la navegación en un pasillo, y muchas están sujetas a un error de medida. Por este motivo, se decidió trabajar únicamente con una submatriz frontal, descartando zonas periféricas menos relevantes y que diesen un error repetitivo.

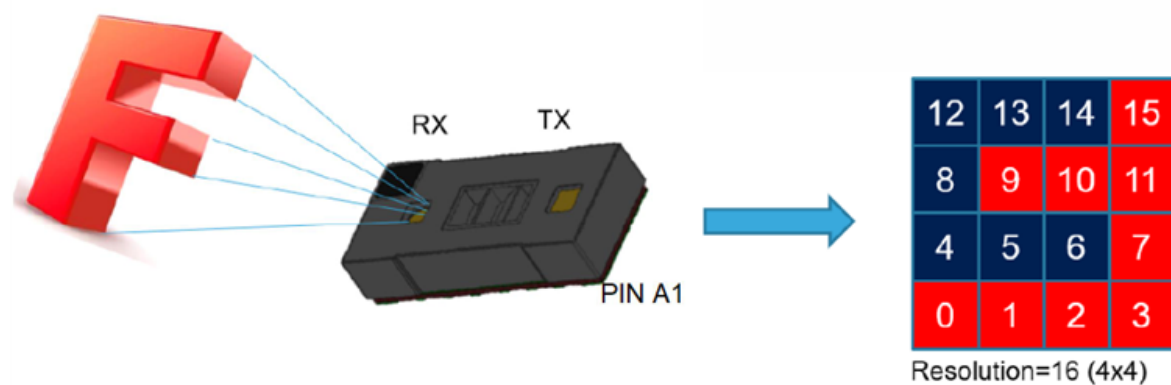


Figure 13. Medida matricial de distancia mediante sensor VL53L5CX.

Selección de celdas y reconstrucción espacial de la matriz

De las 64 celdas disponibles en la matriz 8×8 del sensor, se seleccionó un subconjunto correspondiente a la zona frontal útil para la navegación. Concretamente, se utilizaron

las celdas con índices comprendidos entre 8 y 47, lo que equivale a una submatriz de 5×8 celdas, es decir, 40 mediciones de distancia.

Esta selección no es arbitraria, sino que responde a la necesidad de centrarse en la región del campo de visión que realmente detecta obstáculos frente al rover, evitando las filas superiores e inferiores donde las mediciones son menos relevantes o más propensas a errores. En concreto, se eliminó las dos filas inferiores que aparte de no ser muy fiables, veían el suelo y por tanto no daban una medida adecuada a lo que tenían en frente y la fila superior que tenía mucho error. El ESP32 empaqueta estas 40 distancias junto con sus 40 estados asociados y las envía a MATLAB en una única trama.

En MATLAB, el vector recibido se reconstruye como una matriz de 5×8, que posteriormente se transpone y rota para que la orientación espacial coincida con la disposición real del sensor montado sobre el rover. Esta corrección geométrica es fundamental para que cada columna de la matriz corresponda a una dirección angular concreta respecto al eje de avance del rover.

```
// ----- ToF: enviar 5x8 a Matlab -----
void data_acquisition(){
  if (myImager.getRangingData(&measurementData)){
    int j = 0;
    for (int i = 8; i <= 47; i++) {
      int d = (int)measurementData.distance_mm[i];
      datos[2*j] = (byte)(d / 256);
      datos[2*j + 1] = (byte)(d % 256);

      int st = (int)measurementData.target_status[i];
      datos[80 + 2*j] = (byte)(st / 256);
      datos[80 + 2*j + 1] = (byte)(st % 256);

      j++;
    }
    SerialBT.write(datos,160);
  }
}
```

Figure 14. Selección de celdas útiles.

C56	57	58	59	60	61	62	C63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
C0	1	2	3	4	5	6	C7

C = Corner zones
 INNER = all zones not
 identified as the corner

Figure 15. Matriz inicial.

48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23

Figure 16. Matriz final útil.

```

function Funcion_Recepcion(app,~,~)
    Datos=read(app.bt,160);
    i=1;j=1;
    y=zeros(1,40); % y es el vector de distancia
    s=zeros(1,40); % s es el vector de estado
    while i<=80
        y(j)=256*Datos(i)+Datos(i+1);
        i=i+2; j=j+1;
    end
    i=81; j=1;
    while (i>=81 && i<=160)
        s(j)=256*Datos(i)+Datos(i+1);
        if (s(j) == 5)
            s(j)=1000;
        else
            s(j)=0;
        end
        i=i+2; j=j+1;
    end
    %disp(y);
    Y = reshape(y,[8,5]); % Y es la matriz de distancia 5 x 8
    Y=Y'; % Matriz traspuesta
    Y = rot90(Y,2);

    Y(Y > 800) = 800;

    S = reshape(s,[8,5]); % S es la matriz de estado 8 x 8
    S=S'; % Matriz traspuesta
    S = rot90(S,2);
    %disp(S);
    Yimg = mat2gray(Y,[0 800]); % Transf. Niveles de grises. 0 negro y 800 blanco
    Simg = mat2gray(S,[0 1000]); % Transf. Niveles de grises. 0 negro y 1000 blanco
    imshow(Yimg,'Parent',app.pltOriginal);
    imshow(Simg,'Parent',app.pltEstado);

```

Figure 17. Código de las reconstrucciones espaciales de las matrices en MATLAB.

Matriz de estado (target_status) y su uso en el sistema

Además de la matriz de distancias, el sensor proporciona para cada celda un valor target_status que indica la calidad o validez de la medición. Este valor permite identificar si la distancia medida es fiable o si puede estar afectada por factores como baja reflectancia, ángulos desfavorables o interferencias.

En el sistema desarrollado, esta información se utilizó principalmente con fines de visualización y diagnóstico. En la aplicación MATLAB, los valores de estado se transforman a una representación binaria para su visualización: las celdas con estado

igual a 5 se representan como válidas, mientras que el resto se marcan como inválidas. Esto permite comprobar de forma intuitiva qué regiones del sensor están proporcionando medidas consistentes en cada instante.

Sin embargo, la lógica de decisión del sistema no se basa directamente en el valor de `target_status`. En lugar de descartar medidas únicamente por su estado, se optó por aplicar un filtrado estadístico robusto sobre las distancias, lo que permite eliminar valores erróneos sin perder información potencialmente útil en situaciones límite. De este modo, el `target_status` actúa como una herramienta de análisis visual, mientras que la robustez del control se garantiza mediante técnicas estadísticas.

Target status	Description
0	Ranging data are not updated
1	Signal rate too low on SPAD array
2	Target phase
3	Sigma estimator too high
4	Target consistency failed
5	Range valid
6	Wrap around not performed (Typically the first range)
7	Rate consistency failed
8	Signal rate too low for the current target
9	Range valid with large pulse (may be due to a merged target)
10	Range valid, but no target detected at previous range
11	Measurement consistency failed
12	Target blurred by another one, due to sharpener
13	Target detected but inconsistent data. Frequently happens for secondary targets.
255	No target detected (only if number of target detected is enabled)

Figure 18. Significado de los distintos valores del Target Status.

Comunicación Bluetooth, protocolo de mensajes y gestión del buffer

La comunicación entre MATLAB y el ESP32 se diseñó de forma explícitamente estructurada para garantizar un intercambio de datos robusto, determinista y sin acumulación de información en los buffers, a pesar de no emplear temporizadores ni mecanismos de sincronización externos. Para ello se implementó un protocolo sencillo pero muy estricto, basado en mensajes de longitud fija (2 bytes) y en un esquema de comunicación tipo petición-respuesta (request-response).

Por un lado, el ESP32 envía a MATLAB una trama de datos ToF de tamaño fijo, compuesta por un total de 160 bytes. Esta trama se organiza de forma secuencial: los primeros 80 bytes corresponden a 40 distancias medidas, codificadas cada una en dos bytes (parte alta y parte baja), mientras que los siguientes 80 bytes contienen los 40

valores de estado asociados a esas mismas celdas, igualmente codificados en dos bytes por celda. Esta estructura fija permite que MATLAB reconstruya las medidas de forma inequívoca mediante operaciones del tipo $256 \cdot \text{byte_alto} + \text{byte_bajo}$ y posteriores operaciones de reshape y rotación para recuperar la orientación real del sensor en el rover.

En el lado de MATLAB, la recepción de estas tramas se gestiona mediante un callback configurado explícitamente para activarse únicamente cuando el buffer de recepción contiene exactamente 160 bytes. De este modo, el procesamiento solo se inicia cuando se ha recibido una trama completa, evitando lecturas parciales o desalineadas. Al ejecutar la lectura de esos 160 bytes dentro del callback, el buffer queda completamente vaciado, lo que impide que se acumulen datos antiguos o retrasados.

La comunicación de control desde MATLAB hacia el ESP32 se basa siempre en mensajes de exactamente dos bytes, independientemente del comando concreto. El primer byte identifica el comando y el segundo contiene el dato asociado o un valor nulo cuando no es necesario. Esta decisión de diseño es clave, ya que permite que el ESP32 lea siempre los mensajes de forma determinista comprobando que hay al menos dos bytes disponibles en el buffer antes de leerlos. Aunque en algunos comandos el segundo byte no se utilice, se consume igualmente, evitando que queden restos en el buffer que puedan desalinearse mensajes posteriores.

Los comandos implementados cubren todas las acciones necesarias del sistema: inicio del control y fijación de la referencia inicial de yaw, parada completa del sistema, petición explícita de una nueva lectura ToF, cambio de rumbo relativo en función de la columna seleccionada, ajuste de la velocidad base y retorno a la referencia inicial tras una maniobra de evasión. Esta separación clara de responsabilidades simplifica tanto el código como el razonamiento sobre el estado del sistema.

Un aspecto fundamental del protocolo es el papel del comando de petición de ToF. Este comando actúa como mecanismo de sincronización entre MATLAB y el ESP32. El ESP32 solo envía una trama ToF cuando recibe explícitamente dicho comando. Además, tras enviar los 160 bytes, el programa sale deliberadamente del bucle de lectura de Bluetooth, forzando que el control PID y la actuación sobre los motores se ejecuten antes de atender nuevos comandos entrantes. De este modo se evita que el microcontrolador quede bloqueado procesando mensajes de comunicación y se garantiza que el control del rover tenga prioridad temporal.

Este diseño convierte la comunicación en un bucle cerrado gobernado por MATLAB: MATLAB solicita datos, el ESP32 responde con una única trama, MATLAB procesa la información, toma decisiones y envía los comandos correspondientes, y solo entonces solicita una nueva lectura. Al no existir transmisión continua de datos desde el ESP32, el ritmo global del sistema queda marcado por el tiempo de procesamiento en MATLAB,

lo que elimina el riesgo de saturación del buffer y asegura un tiempo de respuesta controlado y estable.

En resumen, el sistema se estructuró en dos niveles funcionales claramente diferenciados. El nivel de bajo nivel, implementado en el ESP32, se encarga del control en tiempo real, la lectura de sensores y el accionamiento de los motores. El nivel de alto nivel, implementado en MATLAB, gestiona la interpretación del entorno, el filtrado de medidas y la toma de decisiones estratégicas, enviando únicamente referencias de rumbo y velocidad al nivel inferior.

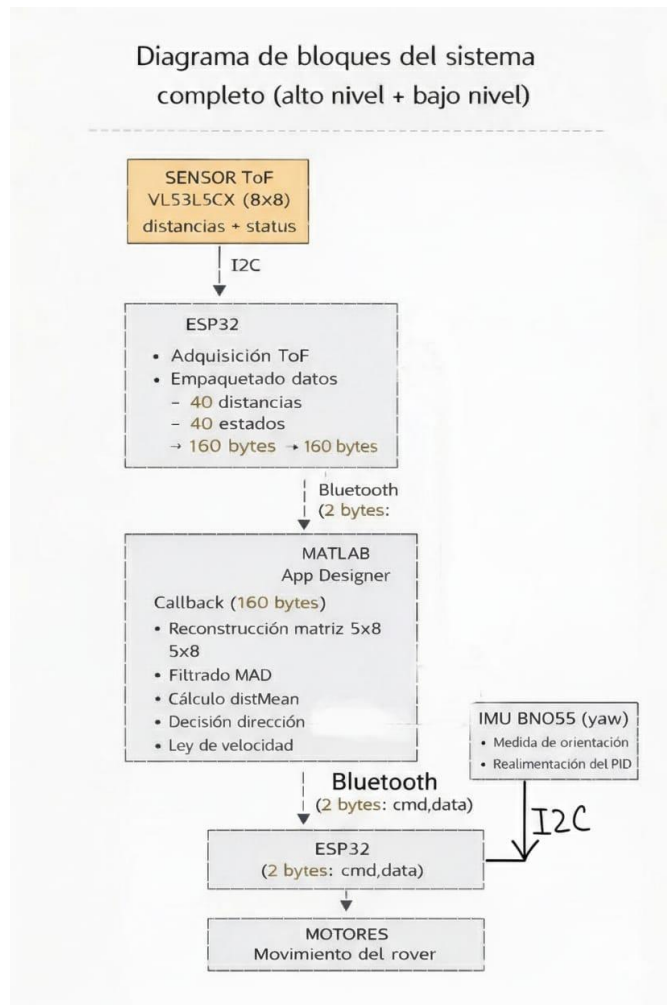


Figure 19. Diagrama de bloques del sistema completo.

La comunicación entre MATLAB y el ESP32 se realiza de forma estrictamente síncrona, siguiendo un esquema de petición–respuesta. MATLAB solicita explícitamente una lectura del sensor ToF mediante el comando cmd=3 y el ESP32 responde enviando una única trama de 160 bytes.

MATLAB no envía nuevas peticiones hasta que dicha trama ha sido completamente recibida y procesada mediante un callback asociado a la llegada exacta de 160 bytes.

Este mecanismo garantiza que el buffer Bluetooth nunca se sature y que no se acumulen datos pendientes de procesar.

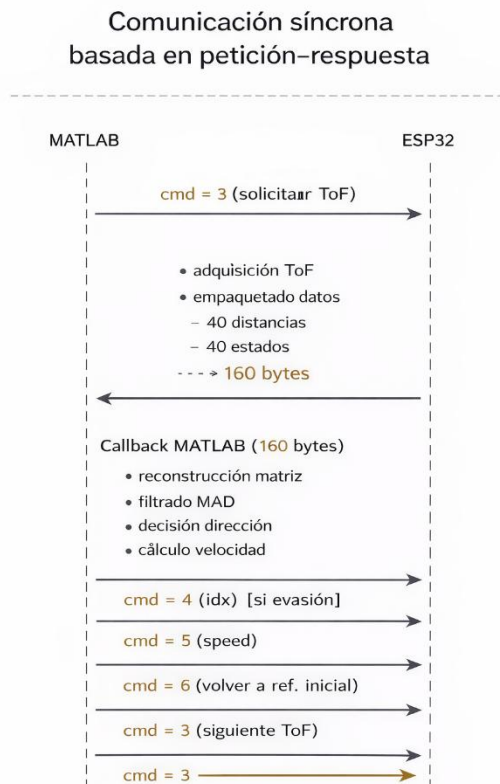


Figure 20. Diagrama Temporal

Table 1. Tabla del protocolo

cmd	data	Dirección	Acción en ESP32
1	0	MATLAB → ESP32	START: lee yaw, guarda ref_yaw_init, fija ref_yaw, pone started=true, resetea PID
2	0	MATLAB → ESP32	STOP: started=false, apaga motores (PWM=0), resetea PID
3	0	MATLAB → ESP32	Solicitud ToF: ESP32 mide y responde con 160 bytes
4	idx(1..8)	MATLAB → ESP32	Evasión: convierte idx a angles_rel[idx-1] y fija ref_yaw = wrap360(yaw_actual + ángulo)
5	speed(0..100)	MATLAB → ESP32	Ajuste de velocidad base: BASE_SPEED = speed

6	0	MATLAB → ESP32	Salida de evasión y vuelta a referencia inicial: vuelve a ref_yaw = ref_yaw_init y resetea PID
---	---	-------------------	--

```

// ----- BT: leer comandos -----
void leerBT() {

  while (SerialBT.available() >= 2) { // SIEMPRE llegan 2 bytes

    uint8_t cmd = (uint8_t)SerialBT.read(); // byte 1
    uint8_t data = (uint8_t)SerialBT.read(); // byte 2

    if (cmd == 1) {
      // START + fijar referencia inicial
      bno.getEvent(sevent);
      X = wrap360(event.orientation.x);

      ref_yaw_init = X; //referencia inicial
      ref_yaw = X;      //
      integral = 0.0;
      prev_error = 0.0;

      started = true;

      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
      digitalWrite(IN3, HIGH);
      digitalWrite(IN4, LOW);
    }

    // ----- STOP -----
    else if (cmd == 2) {
      started = false;

      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
      ledcWrite(ENA, 0);
      ledcWrite(ENB, 0);

      integral = 0.0;
      prev_error = 0.0;
    }

    else if (cmd == 3) {
      // Pedir datos ToF ) )
      data_acquisition();
      delay(10); // espera 10 ms
      break;
    }

    else if (cmd == 4) {
      uint8_t idx = data; // columna

      if (idx >= 1 && idx <= 8) {
        ref_col = idx;
        ref_angle_rel = angles_rel[idx - 1];

        bno.getEvent(sevent);
        X = wrap360(event.orientation.x);
        ref_yaw = wrap360(X + ref_angle_rel);
        integral = 0.0;
        prev_error = 0.0;

        Serial.print("Nueva ref_yaw = ");
        Serial.println(ref_yaw);
      }
    }

    else if (cmd == 6) {
      // Volver a la referencia inicial
      ref_yaw = ref_yaw_init;

      integral = 0.0;
      prev_error = 0.0;

      Serial.println("Volviendo a referencia inicial");
    }

    else if (cmd == 5) {
      // Cambiar velocidad base desde MATLAB (0-100 %)
      BASE_SPEED = constrain(data, 0, 100);
    }
  }
}

```

Figure 21. Código de la función leerBT en Arduino.

Filtrado estadístico de las distancias ToF

Una vez recibida y reconstruida la matriz de distancias, se aplica un filtrado estadístico destinado a eliminar lecturas erróneas sin perder la estructura geométrica real de los obstáculos. Este filtrado se realiza de forma independiente para cada columna del sensor, entendida como un sector angular vertical del campo de visión.

Para cada una de las ocho columnas se toman los cinco valores verticales correspondientes y se calcula su mediana, así como la desviación absoluta de la mediana (MAD). A partir de estas magnitudes se define un umbral basado en un factor k , que en este caso se fijó a 3. Las medidas cuya desviación respecto a la mediana supera ese umbral se consideran outliers y se descartan asignándoles un valor nulo. El resto de valores se consideran válidos y se promedian para obtener una distancia representativa de la columna.

Este enfoque resulta especialmente adecuado en entornos reales, donde pueden aparecer reflexiones puntuales, superficies con baja reflectancia o lecturas erráticas debidas al ángulo de incidencia del haz. Al basarse en estadísticos robustos como la mediana y la MAD, el filtro es poco sensible a valores extremos y conserva la información relevante del entorno sin introducir suavizados excesivos.

```

        k = 3;

        distMean = zeros(1,8);

    for c = 1:8
        x = Y(:,c);

        med = median(x);
        madv = median(abs(x - med));

        x_f = zeros(1,5);
        if madv == 0
            x_f = x;
        else
            for r = 1:5
                variant = abs(x(r) - med);
                if variant <= k*madv
                    x_f(r) = x(r);
                else
                    x_f(r) = 0;
                end
            end
        end

        valid = x_f(x_f ~= 0);
        if isempty(valid)
            distMean(c) = 0;
        else
            distMean(c) = mean(valid);
        end
    end

    [~, idx] = max(distMean);

```

Figure 22. Código del filtrado estadístico.

Extracción de información y decisión de rumbo

A partir del vector de distancias medias por columna obtenido tras el filtrado, el sistema toma decisiones de navegación de forma sencilla y eficiente. En primer lugar, se identifica la columna con mayor distancia libre, interpretada como la dirección con mayor espacio disponible. Esta columna define un posible rumbo de evasión en caso de obstáculo frontal.

Paralelamente, se define una zona frontal de seguridad, formada por las dos columnas centrales del sensor (columnas 4 y 5). De estas columnas se extrae la distancia mínima, que se interpreta como la distancia efectiva al obstáculo más cercano situado frente al rover. Esta magnitud se utiliza como criterio principal para activar o desactivar maniobras de evasión.

Si la distancia frontal cae por debajo de un umbral de seguridad predefinido (600mm), se considera que existe riesgo de colisión y se activa el modo de evasión. En ese caso, MATLAB envía al ESP32 un comando de cambio de rumbo relativo asociado a la columna más libre, haciendo que el controlador PID oriente el rover hacia esa dirección. Cuando la distancia frontal vuelve a superar el umbral y el sistema se encontraba previamente en modo de evasión, se envía un comando para retornar a la referencia inicial, recuperando el rumbo original del pasillo.

Este esquema introduce una memoria mínima de estado que evita oscilaciones continuas entre modos y proporciona transiciones suaves entre la navegación recta y las maniobras de evasión.

```

% ----- VELOCIDAD BASE -----
% zona frontal
d4 = distMean(4);
d5 = distMean(5);

if d4 > 0 && d5 > 0
    d_front = min(d4, d5);
elseif d4 > 0
    d_front = d4;
elseif d5 > 0
    d_front = d5;
else
    d_front = 0;
end

if d_front <= 500
    speed = 0;
elseif d_front >= 700
    speed = 70;
else
    speed = round((d_front - 500) / 200 * 70);
end

persistent avoiding
if isempty(avoiding)
    avoiding = false;
end

if d_front <= 600
    avoiding = true;      % estoy evitando
    write(app.bt, uint8([4 idx]))

elseif d_front >= 600 && avoiding
    write(app.bt, [6 0]) % vuelvo al rumbo inicial
    avoiding = false;    % ya no evito
end

% Enviar velocidad a Arduino
write(app.bt, uint8([5 speed]));

    if app.running
        write(app.bt, uint8([3 0]));
    end

end
end

```

Figure 23. Código de la extracción de información y decisión de rumbo por parte de Matlab.

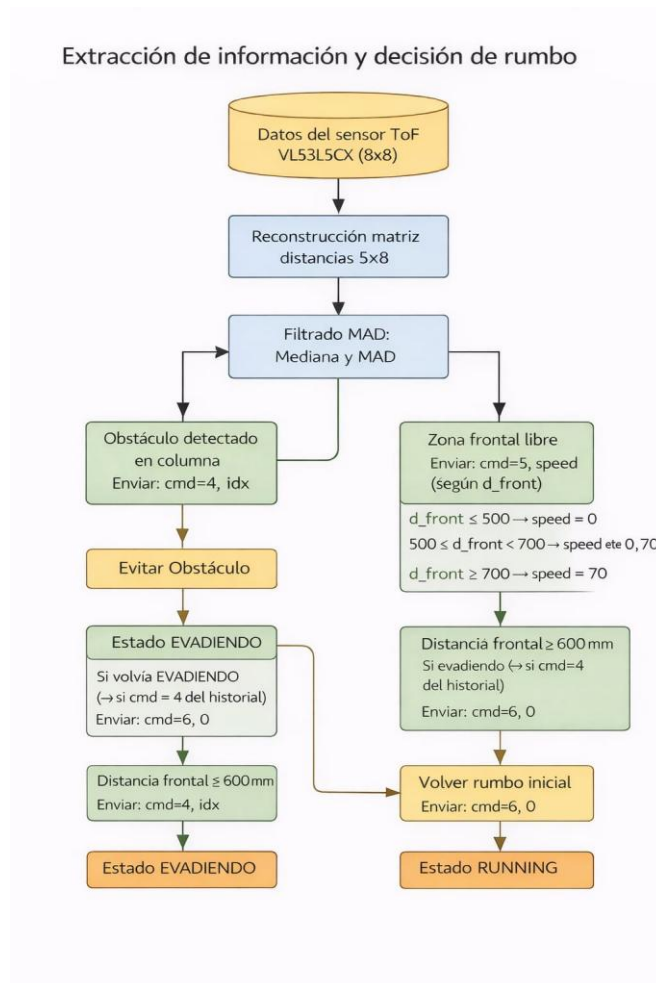


Figure 24. Diagrama de la extracción de información y decisión de rumbo.

Control de velocidad adaptativo

De forma complementaria al control de rumbo, se implementa un control adaptativo de la velocidad longitudinal del rover, con el objetivo de equilibrar seguridad y tiempo de recorrido. La velocidad base se calcula en función de la distancia frontal estimada, de modo que el rover reduce su velocidad progresivamente a medida que se aproxima a un obstáculo.

Cuando la distancia frontal es muy pequeña, la velocidad se reduce a cero, deteniendo el rover para evitar colisiones. Cuando la distancia es suficientemente grande, se permite una velocidad alta y constante. Entre ambos extremos se define una banda intermedia en la que la velocidad se interpola linealmente en función de la distancia, generando transiciones suaves y evitando frenadas bruscas.

Este enfoque reduce significativamente el riesgo de impactos y penalizaciones en competición, al tiempo que permite mantener una velocidad elevada cuando el entorno es despejado. Además, al enviar la velocidad base como un comando independiente al ESP32, se desacopla el control longitudinal del control de rumbo, simplificando el diseño del controlador PID.

```

% ----- VELOCIDAD BASE -----
% zona frontal
d4 = distMean(4);
d5 = distMean(5);

if d4 > 0 && d5 > 0
    d_front = min(d4, d5);
elseif d4 > 0
    d_front = d4;
elseif d5 > 0
    d_front = d5;
else
    d_front = 0;
end

if d_front <= 500
    speed = 0;
elseif d_front >= 700
    speed = 70;
else
    speed = round((d_front - 500) / 200 * 70);
end

% Enviar velocidad a Arduino
write(app.bt, uint8([5 speed]));

```

Figure 25. Código del control de velocidad adaptativo.

Control de rumbo en el ESP32 mediante IMU y PID

El control de rumbo del rover se implementa íntegramente en el ESP32 y constituye el nivel más bajo del sistema de navegación, siendo el encargado directo de generar el movimiento físico del vehículo. Este control se basa en la lectura de orientación proporcionada por la IMU BNO055 y en un regulador PID que actúa sobre la velocidad diferencial de los motores.

La IMU BNO055 proporciona directamente el ángulo de yaw absoluto del rover en el rango de 0 a 360 grados. Para evitar problemas de discontinuidad angular al cruzar el límite 0/360, el cálculo del error de orientación se realiza en un dominio angular entre -180 y +180 grados. Para ello, la diferencia entre el rumbo de referencia y el yaw actual se procesa mediante una función de normalización que garantiza que el error siempre represente la rotación mínima necesaria para corregir la orientación.

A partir de este error angular se ejecuta el controlador PID. El término proporcional actúa como principal responsable de corregir desviaciones de rumbo, mientras que el término integral permite compensar errores persistentes debidos a asimetrías mecánicas o diferencias entre motores. El término derivativo se fijó a cero, ya que en las pruebas experimentales no aportó ninguna mejora y tendía a amplificar el ruido de

la señal de orientación, generando una oscilación constante. Para asegurar un comportamiento estable, tanto la integral como la salida total del controlador se limitan mediante saturaciones explícitas, evitando fenómenos de wind-up y comandos excesivos.

La salida del PID se traduce directamente en una corrección diferencial de velocidad entre las ruedas. La velocidad base, calculada en MATLAB en función de la distancia frontal, se ajusta sumando la corrección a un motor y restándola al otro. De este modo, el rover mantiene su velocidad longitudinal mientras corrige el rumbo de forma suave y continua. Finalmente, las velocidades resultantes se saturan al rango permitido por el hardware y se convierten a señales PWM para accionar los motores.

```
const float angles_rel[8] = {
    -24.0625, -17.1875, -10.3125, -3.4375,
    3.4375, 10.3125, 17.1875, 24.0625
}; // angulos de yaw respecto a cada columna del sensor

// ----- funciones ángulos -----
float wrap360(float a) {
    while (a >= 360.0) a -= 360.0;
    while (a < 0.0) a += 360.0;
    return a;
}

float wrap180(float a) {
    while (a > 180.0) a -= 360.0;
    while (a < -180.0) a += 360.0;
    return a;
}
```

```

void loop() {

    // 1) Leer BT (Matlab pide datos o manda ref)
    leerBT();

    if (!started) {
        delay(5);
        return;
    }

    // 2) Leer IMU
    bno.getEvent(&event);
    X = wrap360(event.orientation.x);

    // 3) Error respecto a la referencia absoluta
    error = wrap180(ref_yaw - X);

    // 4) PID
    T = micros();
    delta_T = (float)(T - prev_T) / 1e6;
    prev_T = T;

    integral += error * delta_T;
    if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
    if (integral < -INTEGRAL_MAX) integral = -INTEGRAL_MAX;

    derivator = (error - prev_error) / delta_T;
    prev_error = error;

    u = Kp * error + Ki * integral + Kd * derivator;
    if (u > U_MAX) u = U_MAX;
    if (u < -U_MAX) u = -U_MAX;

    // 5) Motores
    float leftSpeed = BASE_SPEED + u;
    float rightSpeed = BASE_SPEED - u;

    if (leftSpeed < 0) leftSpeed = 0;
    if (leftSpeed > 100) leftSpeed = 100;
    if (rightSpeed < 0) rightSpeed = 0;
    if (rightSpeed > 100) rightSpeed = 100;

    digitalWrite(IN1, LOW);
}

```

Figure 26. Código del control PID en Arduino.

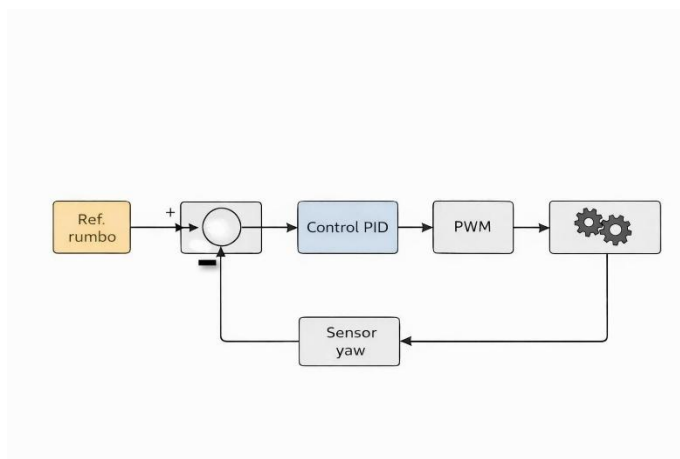


Figure 27. Diagrama de bloques del bucle de control

Definición y gestión de referencias de rumbo

Uno de los aspectos más característicos del sistema desarrollado es la forma en que se definen y gestionan las referencias de rumbo del rover. En lugar de trabajar únicamente con referencias absolutas fijas, se emplea un esquema híbrido que combina una referencia inicial absoluta con referencias relativas asociadas a las columnas del sensor ToF. Este planteamiento permite mantener un rumbo global coherente dentro del pasillo, al mismo tiempo que se introducen desviaciones controladas y temporales para la evitación de obstáculos.

Cuando se activa el sistema mediante el comando de inicio, el ESP32 lee el valor actual de yaw proporcionado por la IMU y lo almacena simultáneamente como referencia inicial y como referencia activa de control. Esta referencia inicial representa el rumbo “recto” del pasillo en el instante de arranque y se conserva durante toda la ejecución como punto de retorno tras las maniobras de evasión. De forma simultánea, se reinician las variables internas del controlador PID, garantizando una respuesta limpia y predecible desde el primer instante de control.

Durante una maniobra de evitación, el procesamiento de la información ToF se realiza en MATLAB. A partir de la matriz de distancias filtradas, se identifica la columna que presenta mayor espacio libre y se envía al ESP32 el índice de dicha columna. Este índice no representa directamente un ángulo, sino una región espacial del campo de visión del sensor. El ESP32 traduce entonces este índice en un ángulo relativo predefinido mediante una tabla que asocia cada columna del sensor con un desplazamiento angular respecto al eje frontal del rover.

La nueva referencia de rumbo se calcula sumando este ángulo relativo al yaw actual medido por la IMU en ese instante. De este modo, la referencia se define siempre de forma absoluta en el sistema global, pero calculada a partir de la orientación real instantánea del vehículo, evitando acumulaciones de error o giros basados en referencias que no son correctas. Esta referencia absoluta es la que se introduce en el controlador PID, que se encarga de corregir el rumbo mediante una acción de velocidad diferencial en los motores.

Una vez superado el obstáculo y cuando la distancia frontal vuelve a ser segura, MATLAB ordena explícitamente el retorno al rumbo inicial. El ESP32 recupera entonces la referencia inicial almacenada al inicio de la ejecución y el PID vuelve a corregir el rumbo hacia dicha orientación, permitiendo que el rover se recoloque automáticamente en el eje del pasillo sin intervención adicional.

Para poder realizar esta conversión entre columnas del sensor y ángulos de giro relativos de forma coherente, fue necesario establecer una correspondencia geométrica basada en el campo de visión real del sensor VL53L5CX. El sensor presenta un campo de visión horizontal aproximado de 55 grados, que se distribuye de forma uniforme entre las 8 columnas de la matriz de distancias. Por tanto, cada columna cubre un sector angular de aproximadamente $55/8 \approx 6,875$ grados. Dado que el eje central del sensor coincide con la dirección frontal del rover, el campo de visión queda centrado alrededor de 0 grados, abarcando aproximadamente desde $-27,5^\circ$ hasta $+27,5^\circ$.

Para cada columna no se tomó el valor correspondiente a los bordes del sector angular, sino el ángulo asociado al centro geométrico de cada columna. Esta elección permite representar de forma más fiel la dirección media del espacio libre detectado en dicha columna y evita giros excesivos asociados a los límites del campo de visión. De este modo, cada columna se asocia a un ángulo relativo que apunta al centro de su región angular, proporcionando un compromiso adecuado entre precisión direccional y suavidad en la maniobra de evasión.

A partir de este criterio se definió un vector de ángulos relativos, donde las columnas más a la izquierda se asocian a ángulos negativos y las más a la derecha a ángulos positivos, manteniendo una distribución simétrica respecto al eje frontal del rover. Estos ángulos representan el desplazamiento angular que debe aplicarse al yaw actual para orientar el vehículo hacia la zona más despejada detectada por el sensor y constituyen el nexo entre la percepción espacial del ToF y el control de rumbo basado en la IMU y el PID.

```
const float angles_rel[8] = {
  -24.0625, -17.1875, -10.3125, -3.4375,
  3.4375, 10.3125, 17.1875, 24.0625
}; // angulos de yaw respecto a cada columna del sensor
```

Figure 28. Vector de ángulos relativos en Arduino.

Columna	Ángulo relativo (°)
1	-24.06
2	-17.19
3	-10.31
4	-3.44
5	3.44
6	10.31
7	17.19
8	24.06

Figure 29. Tabla de los ángulos relativos correspondientes a cada columna del sensor.

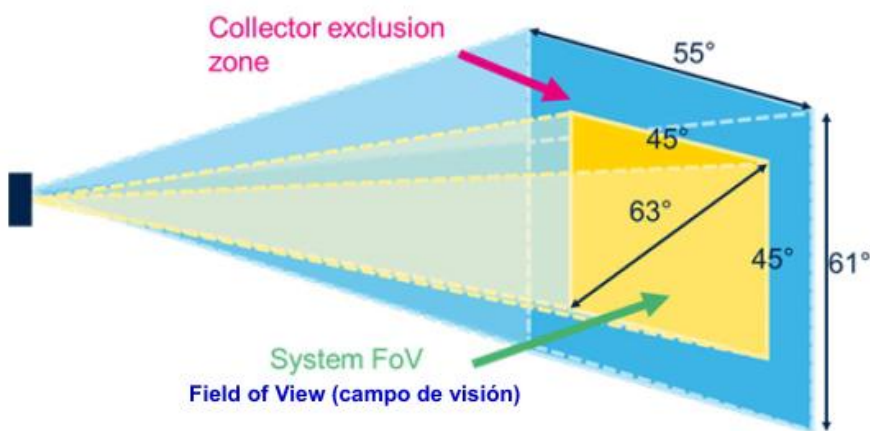


Figure 30. Campo de visión del sensor.

Máquina de estados global del sistema

El funcionamiento completo del sistema puede entenderse como una máquina de estados distribuida entre MATLAB y el ESP32, donde ambos elementos mantienen estados coherentes y coordinados mediante el protocolo Bluetooth.

En MATLAB, el botón de inicio activa el sistema estableciendo la variable de ejecución y enviando simultáneamente el comando de arranque y la primera petición de datos ToF.

A partir de ese momento, cada vez que se recibe una trama completa de 160 bytes, MATLAB procesa la información, toma decisiones de navegación y, si el sistema sigue activo, solicita explícitamente una nueva lectura. De este modo, el flujo de datos queda completamente controlado por MATLAB y no se generan lecturas innecesarias.

Dentro de MATLAB se mantiene además una variable de estado que indica si el sistema se encuentra en modo de evasión o en navegación normal. Esta variable evita cambios bruscos de decisión y asegura que las órdenes de retorno al rumbo inicial solo se envíen cuando realmente se ha salido de la situación de peligro, que corresponde con una distancia frontal mayor a 600mm.

En el ESP32, una variable lógica determina si el controlador PID está activo o no. Mientras el sistema no ha sido iniciado o se encuentra en estado de parada, los motores permanecen completamente apagados, independientemente de que se reciban otros comandos. Solo cuando el estado de ejecución está activo se leen la IMU, se calcula el PID y se actúa sobre los motores. Esta separación clara de estados garantiza un comportamiento seguro y fácilmente interpretable.

Ajustes prácticos y decisiones de ingeniería

Durante el desarrollo del sistema se tomaron diversas decisiones prácticas orientadas a mejorar la robustez y fiabilidad del sistema. Una de ellas fue limitar las distancias máximas consideradas a un valor fijo, correspondiente al rango útil real del sensor en el entorno del pasillo (800mm). Este capado evita saturaciones visuales en la interfaz y reduce la influencia de lecturas lejanas poco relevantes para la navegación inmediata.

Asimismo, los umbrales utilizados para la toma de decisiones se ajustaron experimentalmente. Se definieron distintos niveles de distancia para detener el rover, activar la evasión y permitir velocidad máxima, buscando un equilibrio entre seguridad y rapidez. Estos valores se validaron mediante pruebas repetidas en el entorno real, observando el comportamiento del rover frente a los obstáculos.

La frecuencia efectiva de funcionamiento del sistema viene determinada por la combinación de la frecuencia de medición del sensor ToF y el tiempo de procesamiento en MATLAB. Al trabajar con peticiones explícitas y callbacks, se obtiene una frecuencia real cercana a la nominal del sensor, sin necesidad de temporizadores adicionales y sin riesgo de saturar la comunicación.

Durante las pruebas se observó también que ciertas zonas del campo de visión del sensor resultaban menos fiables, especialmente en los extremos, debido a efectos geométricos y de reflectancia. Por este motivo, se priorizó el uso de una submatriz central y se reforzó el filtrado estadístico para minimizar la influencia de estas zonas.

Target reflectance level. Full FoV (reflectance %)	Zone	Dark	Ambient light (5 klux)
White target (88 %)	Inner	Typical: 4000 mm Minimum: 4000 mm	Typical: 1700 mm Minimum: 1400 mm
	Corner	Typical: 4000 mm Minimum: 4000 mm	Typical: 1400 mm Minimum: 1100 mm
Grey target (17 %)	Inner	Typical: 2400 mm Minimum: 1900 mm	Typical: 1000 mm Minimum: 900 mm
	Corner	Typical: 2200 mm Minimum: 1800 mm	Typical: 950 mm Minimum: 850 mm

Figure 31. Resultados experimentales de la visibilidad del sensor VL53L5CX.

Validación experimental y resultados

La validación del sistema se realizó mediante múltiples pruebas en el entorno de competición. Se consideró el número de colisiones, el tiempo total de recorrido y la suavidad de las maniobras de evasión.

Durante las pruebas se observó que el sistema activaba de forma consistente las maniobras de evasión cuando la distancia frontal descendía por debajo del umbral establecido, orientándose hacia la zona más despejada y recuperando posteriormente el rumbo original sin oscilaciones apreciables. El control de velocidad adaptativo permitió reducir significativamente los impactos, manteniendo al mismo tiempo una velocidad elevada cuando el entorno lo permitía.

Cabe destacar que los códigos completos finales de la última prueba tanto de Matlab como Arduino se encuentran en la carpeta de los archivos adjuntados junto con esta memoria y no se han subido aquí para ahorrar espacio.

4. Mejoras Proyecto Rover

En este apartado se analizan posibles mejoras a los códigos presentados y al rover.

En primer lugar, los códigos de la primera y segunda competición son muy completos y permiten al usuario controlar de manera muy eficaz en el caso de la primera competición y ser capaz de reiniciar su sistema de referencia en el caso de la segunda.

Como posible mejora, se podría usar otro tipo de aplicación distinta a Kodular o incluir una función que contenga un joystick para poder hacerlo mas manejable en todas las direcciones.

Por el contrario, respecto a la tercera competición, si que se podrían hacer mejoras significativas en el código . Una opción que tuvimos en cuenta sería implementar la marcha atrás cuando el rover se encontrase muy cerca del obstáculo que tiene delante. Además, se podría hacer un estudio en ese mismo momento analizando la vía con menos obstáculos que puede detectar con el sensor sin perder la referencia. Esto se

intentó implementar sin éxito en esta competición, pero no se pudo lograr con precisión.

Cabe destacar también respecto a la tercera competición que, como limitaciones, se identificó que la estrategia de seleccionar únicamente la columna más libre no tiene en cuenta la continuidad temporal del entorno ni la evolución futura del obstáculo. Como posibles mejoras futuras, se podría introducir un margen de tolerancia en los umbrales para evitar cambios bruscos de decisión, emplear métodos de planificación más completos que tengan en cuenta varias columnas al mismo tiempo, y añadir algún tipo de suavizado temporal que haga más estables las decisiones de rumbo y velocidad.

Por otro lado aunque el rover cuenta actualmente con un robusto control PID de orientación basado en la IMU (que asegura que avance en línea recta), una mejora significativa sería la incorporación de encoders en los motores. Esto permitiría expandir el sistema de control actual para incluir odometría, es decir, el robot no solo sabría hacia dónde mira, sino exactamente qué distancia ha recorrido, permitiendo una navegación precisa por coordenadas (X, Y) en lugar de por tiempos aproximados.

Otra vía de evolución sería aprovechar la conectividad Wi-Fi del ESP32 para crear una interfaz IoT (Internet of Things). En lugar de depender de una aplicación Bluetooth específica, el rover podría generar su propia página web local donde mostrar el mapa de obstáculos en tiempo real y permitir el control desde cualquier dispositivo (móvil, tablet o PC) sin necesidad de instalar software adicional, e incluso integrar una pequeña cámara para conducción remota (FPV).

5. Fuentes utilizadas para el desarrollo del proyecto

La información necesaria para el desarrollo de los códigos y del sistema de control del rover se obtuvo principalmente a partir de los conocimientos adquiridos durante la asignatura y de la consulta de documentación técnica oficial.

En primer lugar, se emplearon los conceptos teóricos vistos en clase relacionados con sistemas de control en lazo cerrado, reguladores PID, tratamiento de señales y control de motores. Estos conocimientos sirvieron como base para el diseño del control de rumbo del rover, la implementación del regulador PID en el microcontrolador y la estructuración general del sistema.

Asimismo, se consultaron las hojas de datos y guías de uso oficiales del sensor de distancia VL53L5CX. Esta documentación fue fundamental para comprender el principio de funcionamiento del sensor Time of Flight, su configuración en modo matricial, la interpretación de las distancias medidas y de los valores de estado asociados, así como las limitaciones prácticas del sensor en entornos reales.

Finalmente, una parte importante de la información se obtuvo mediante la experimentación directa con el sistema. A través de pruebas repetidas se validaron los parámetros de funcionamiento, se ajustaron los umbrales de decisión y se evaluó el comportamiento del rover en condiciones reales, permitiendo adaptar el código y la estrategia de control a las necesidades del proyecto.

6. Conclusiones

El desarrollo de este proyecto ha permitido aplicar de forma práctica y progresiva los conocimientos adquiridos a lo largo de la asignatura, integrando conceptos de electrónica, control automático, sensórica y programación en un sistema robótico real. A través de las distintas competiciones se ha podido comprobar cómo un mismo hardware puede evolucionar desde un control manual sencillo hasta un sistema de navegación autónoma con toma de decisiones, lo que ha supuesto una experiencia formativa completa y altamente representativa de un entorno de ingeniería real.

En la primera competición se logró implementar con éxito un sistema de control manual robusto y flexible mediante una aplicación móvil desarrollada en Kodular y comunicación Bluetooth con el ESP32. Este primer enfoque permitió validar el correcto funcionamiento del hardware, del control de motores y de la comunicación inalámbrica, sirviendo además como base sólida para el resto del proyecto. La evolución hacia un control basado en potencia longitudinal y giro mejoró notablemente la manejabilidad del rover, demostrando la importancia de una correcta abstracción del control desde el punto de vista del usuario.

La segunda prueba permitió introducir técnicas de control automático mediante la implementación de un regulador PID de orientación basado en la información proporcionada por la IMU. Este sistema permitió corregir de forma autónoma las desviaciones del rover al avanzar, compensando asimetrías mecánicas y perturbaciones externas. El ajuste experimental de las ganancias y la decisión de prescindir de la acción derivativa pusieron de manifiesto la importancia del análisis práctico y del compromiso entre estabilidad y rapidez de respuesta en sistemas reales con sensores ruidosos.

En la tercera competición se abordó un problema de mayor complejidad, combinando navegación autónoma, detección de obstáculos y toma de decisiones en tiempo real. La separación clara entre un nivel de alto nivel implementado en MATLAB y un nivel de bajo nivel ejecutado en el ESP32 permitió estructurar el sistema de forma ordenada y escalable. El uso de un protocolo Bluetooth síncrono y determinista evitó problemas de saturación de buffers y garantizó un comportamiento estable durante toda la ejecución, incluso sin el uso de temporizadores adicionales.

El empleo del sensor de distancia matricial ToF junto con técnicas de filtrado estadístico robusto permitió obtener información fiable del entorno y tomar decisiones de rumbo y velocidad de manera eficaz. La combinación de referencias de rumbo absolutas y relativas, junto con el control PID basado en la IMU, proporcionó un sistema de evasión de obstáculos suave, coherente y estable, capaz de recuperar el rumbo original tras superar situaciones de riesgo.

En conjunto, los resultados obtenidos durante las pruebas experimentales validan el correcto funcionamiento del sistema desarrollado y demuestran que los objetivos planteados al inicio del proyecto se han cumplido satisfactoriamente. Además de los logros técnicos alcanzados, este proyecto ha permitido desarrollar una visión más completa del diseño de sistemas electrónicos integrados y de control, destacando la importancia de la integración entre hardware, software y estrategias de control para resolver problemas reales de ingeniería.